National University of Computer and Emerging Sciences, Lahore

Growise: AI-Powered Dynamic Learning Platform

Muhammad Umair Imran 22L-8370 BS(DS)

Muhammad Shahzad Waris 22L-7530 BS(DS)

Ameer Tufail 22L-7530 BS(DS)

Supervisor: Maryam Nasim

Final Year Project

April 23, 2026

Anti-Plagiarism Declaration

This is to declare that the above publication was produced under the:

Title: Growise: AI-Powered Dynamic Learning Platform

is the sole contribution of the author(s), and no part hereof has been reproduced as it is the basis (cut and paste) that can be considered Plagiarism. All referenced parts have been used to argue the idea and cited properly. I/We will be responsible and liable for any consequence if a violation of this declaration is determined.

Date: 14/10/2025

Name: Muhammad Umair Imran

umair

Name: Muhammad Shahzad Waris

shahzad

Name: Ameer Tufail

ameer

Author's Declaration

This states Authors' declaration that the work presented in the report is their own, and has not been submitted/presented previously to any other institution or organization.

Abstract

This project gives a smart learning platform for programmers. It helps them get real problem-solving skills. It does not let them only remember code without thinking. The system first looks at what the learner knows now and his technical past. Then it makes a personal learning road that fits only him. During all the learning time, an AI helper gives help right away, explains things, and brings good materials to make understanding better. After every small part, learners get tested with real coding work that is same as what big company engineers do every day. By mixing learning that changes for each person, non-stop feedback, and real hand practice, the platform helps learners grow true programming power and makes them much more ready for real jobs in software making.

Executive Summary

Today tech world change very very fast. Developer must learn new tool and new way super quick. But old school programming course not good. They only make student remember code word and follow same long plan for everybody. This waste time a lot because some people already know some thing and some want different job. This old way make people not ready for job and they feel bored and stop learn.

So we make one smart learning platform. It teach you how to think like real programmer and how to solve problem, not just remember code like robot. First day the platform give you small test. It check what you know already and what you good at. Then it ask you want learn web or AI or what. After that it make one special plan only for you. You only study what you really need. No waste time.

All the time you study, one AI teacher stay with you. It explain hard thing with easy word. It answer your question fast. It give you video or article that help you right now. After each lesson, you complete projects that feel like real work situations. These projects test whether you understand the concepts and can make good decisions about how to solve problems. You can track your progress on a dashboard that shows you what you're good at, what you need to work on, and how to improve.

By moving away from long, one-size-fits-all courses to a flexible, problem-solving focused approach, this platform bridges the gap between school and real jobs. It helps you grow faster as a developer, teaches you how to think through problems step by step, and gives you skills you can actually use at work. The platform keeps up with how modern software development actually works.

Table of Contents

List of Figures **xi**

List of Tables **xii**

1 Introduction **1**

1.1 Purpose of this Document 1

1.2 Intended Audience 1

1.3 Definitions, Acronyms, and Abbreviations 2

1.4 Conclusion 2

2 Project Vision **3**

2.1 Problem Domain Overview 3

2.2 Problem Statement 3

2.3 Problem Elaboration 3

2.4 Goals and Objectives 4

2.5 Project Scope 4

2.6 Sustainable Development Goal (SDG) 6

2.7 Constraints 6

2.8 Business Opportunity 7

2.9 Stakeholders Description/ User Characteristics 7

2.9.1 Stakeholders Summary 7

2.9.2 High-Level Goals and Problems of Stakeholders 7

2.10 Conclusion 8

3 Literature Review / Related Work **9**

3.1 Definitions, Acronyms, and Abbreviations 9

3.1.1 Key Definitions 9

3.2 Detailed Literature Review 9

3.2.1 Crio.Do 10

3.2.2 The Odin Project . . . 10
3.2.3 ProjectLearn.io . . . 11
3.2.4 Educative.io . . . 11
3.2.5 Codio . . . 12
3.2.6 Project-Based Learning GitHub Repository . . . 12
3.2.7 Udemy . . . 13
3.2.8 LinkedIn Learning . . . 14
3.2.9 Microsoft Learn . . . 14
3.2.10 Pluralsight . . . 15
3.2.11 CYPHER Learning (with CYPHER Agent) . . . 15
3.2.12 DataCamp . . . 16
3.2.13 freeCodeCamp . . . 17
3.2.14 Udacity . . . 17
3.2.15 CodeCombat . . . 18
3.3 Literature Review Summary Table . . . 18
3.4 Conclusion . . . 20

4 Software Requirement Specifications 21
4.1 List of Features . . . 21
4.2 Functional Requirements . . . 21
4.2.1 Functional Requirements of Programmers / Developers . . . 21
4.2.2 Functional Requirements of Admin . . . 22
4.2.3 Functional Requirements of System . . . 22
4.3 Quality Attributes . . . 22
4.4 Non-Functional Requirements . . . 23
4.4.1 Performance . . . 23
4.4.2 Reliability . . . 23
4.4.3 Usability . . . 23
4.4.4 Reusability . . . 23
4.4.5 Extensibility . . . 23
4.5 Assumptions . . . 23
4.6 Use Cases . . . 24
4.6.1 User Login . . . 24
4.6.2 Track Selection . . . 25
4.6.3 Skill Assessment . . . 26

4.6.4 AI Path Generation . . . 27
4.6.5 Agentic RAG Assistance . . . 28
4.6.6 Project-Based Assignment . . . 29
4.6.7 Admin Login . . . 29
4.6.8 Manage Learning Paths . . . 30
4.6.9 Manage RAG Knowledge Base . . . 30
4.6.10 Monitor User Progress . . . 31
4.6.11 Learning Path Module Access . . . 32
4.7 Hardware and Software Requirements . . . 32
4.7.1 Hardware Requirements . . . 32
4.7.2 Software Requirements . . . 32
4.8 Graphical User Interface . . . 33
4.9 Database Design (if required; this means if you are using noSQL, you will not provide ER but the other design element and data dictonary should still be there. Explain and elaborate your DB design) . . . 36
4.9.1 ER Diagram . . . 36
4.9.2 Data Dictionary . . . 38
4.10 Risk Analysis . . . 46
4.10.1 Technical Risks . . . 46
4.10.2 Business Risks . . . 46
4.10.3 User Risks . . . 47
4.10.4 Security Risks . . . 47

5 High-Level and Low-Level Design 48
5.1 System Overview . . . 48
5.1.1 Learning Management and Modules . . . 48
5.1.2 Personalized Learning Paths . . . 48
5.1.3 Agentic RAG Mentor (Context-Aware Assistance) . . . 48
5.1.4 Assessment and AI Software Engineer Evaluation . . . 48
5.1.5 Progress Insights and Feedback . . . 48
5.1.6 High-Level Technology View . . . 49
5.2 Design Considerations . . . 49
5.2.1 Assumptions and Dependencies . . . 49
5.2.2 General Constraints . . . 50
5.2.3 Goals and Guidelines . . . 50

5.2.4 Development Methods 50
5.3 System Architecture 51
5.3.1 High-Level System Decomposition 52
5.3.2 Major System Responsibilities 52
5.3.3 Component Roles and Responsibilities 52
5.3.4 Component Collaboration and Data Flow 53
5.3.5 Architectural Rationale 54
5.3.6 External System Integration 54
5.3.7 Subsystem Architecture 54
5.4 Architectural Strategies 55
5.4.1 Using External AI APIs 55
5.4.2 Using RAG for Smart Help 55
5.4.3 Choosing Modern Technology 56
5.4.4 Using Microservices 56
5.4.5 API-First Design 56
5.5 Domain Model/Class Diagram 56
5.6 Policies and Tactics 58
5.6.1 Codebase Organization 58
5.6.2 Development Tools 59
5.6.3 Coding Guidelines 59
5.6.4 Testing Strategies 59
5.6.5 Documentation Practices 59
5.7 Conclusion 59

6 Implementation and Testing **61**
6.1 Implementation 61
6.1.1 Learning Track and Module System 61
6.1.2 AI-Powered Skill Assessment Module 61
6.1.3 AI Learning Path Generation Module 62
6.1.4 RAG Implementation for Integrated Learning Sources Context 62
6.1.5 AI Evaluation Module 63
6.2 Test case Design and description 63
6.3 Test Metrics 71
6.3.1 Test Metric No.1: Overall Test Summary 72
6.3.2 Test Metric No.2: Module-Wise Test Execution Results 72

6.3.3 Test Metric No.3: Requirements Traceability Matrix . . . 73
6.3.4 Conclusion . . . 73

7 User Manual **74**
7.1 Introduction . . . 74
7.2 System Requirements . . . 74
7.3 Accessing the Platform . . . 74
7.4 Sign Up . . . 74
7.5 Login . . . 75
7.5.1 Forgot Password . . . 75
7.5.2 Reset Password . . . 76
7.6 Selecting a Learning Track . . . 76
7.7 Skill Assessment . . . 77
7.8 Dashboard . . . 78
7.8.1 Stat Cards . . . 78
7.8.2 Activity Timeline . . . 78
7.8.3 Assessment History and AI Report . . . 78
7.8.4 Comparing Two Assessments . . . 79
7.8.5 Path Completion Banner . . . 79
7.9 Learning Path . . . 79
7.9.1 Navigating Stages . . . 79
7.9.2 Generating Stage Content . . . 79
7.9.3 Tracking Your Progress . . . 80
7.9.4 Using the AI Mentor Chat . . . 80
7.9.5 Completing the Learning Path . . . 81
7.10 AI Evaluation (Validator Interview) . . . 81
7.10.1 Starting a New Session . . . 81
7.10.2 Conducting the AI Interview . . . 81
7.10.3 Completing the Evaluation . . . 82
7.11 Evaluation Report . . . 82
7.12 Improvement Analysis . . . 83
7.13 Account and Security Settings . . . 84
7.13.1 Updating Your Profile . . . 84
7.13.2 Changing Your Password . . . 84
7.13.3 Managing Active Sessions . . . 84

7.13.4 Deleting Your Account 85
7.14 Logging Out 85
7.15 Conclusion 85

8 Conclusions **86**
8.1 Project Summary and Achievements 86
8.2 Key Findings and Results 86
8.3 Project Scope Coverage and Objectives Achievement 86
8.4 Challenges Faced and Limitations 86
8.5 Long-term Recommendations 87
8.6 Final Remarks 87

References **88**

List of Figures

2.1 SDG 4 Quality Education, the goal GrowWise aligns with. 6

4.1 Login screen for users to enter credentials and access the system. 33
4.2 Track selection screen where users choose their learning path. 34
4.3 Skill assessment page for evaluating users' technical and problem-solving abilities. . . 34
4.4 Assessment report showing the user's skill level and performance summary. 34
4.5 Personalized roadmap showing the AI-generated learning journey and module sequence. 35
4.6 Learning module interface with study content and an integrated AI chatbot. 35
4.7 AI software engineer screen that assigns projects and evaluates user responses. . . . 35
4.8 Insights dashboard with user progress analytics. 36
4.9 Advanced insights with AI evaluation and performance graphs. 36
4.10 GrowWise Entity-Relationship Diagram with all entities, keys, and relationships. . . 37

5.1 GrowWise architecture overview—showing interaction between frontend, backend, AI services, and database. 51
5.2 GrowWise system architecture: frontend, backend, AI, and database. 57

5.3 GrowWise – View 2: AI Services & Knowledge Management, highlighting components like the RAG knowledge base, contextual assistant, path generator, and AI mentor for adaptive, context-aware learning. 57
5.4 GrowWise – View 3: Project Workflow & Evaluation, showing how projects are assigned, submitted, and AI-evaluated with scoring and feedback. 58

7.1 Dashboard showing the assessment report and progress overview. 78

List of Tables

3.1 Summary of Learning Platforms Reviewed 19

4.1 Use Case: User Login 24
4.2 Use Case: Track Selection 25
4.3 Use Case: Skill Assessment 26
4.4 Use Case: AI Path Generation 27
4.5 Use Case: RAG AI Assistance 28
4.6 Use Case: Project Assignment 29
4.7 Use Case: Admin Login 29
4.8 Use Case: Learning Path Management 30
4.9 Use Case: RAG Knowledge Management 30
4.10 Use Case: User Progress Monitoring 31
4.11 Use Case: Module Access 32
4.12 Test case for User 39
4.13 Test case for LearningTrack 39
4.14 Test case for LearningModule 40
4.15 Test case for UserTrackSelection 40
4.16 Test case for UserProgress 41
4.17 Test case for SkillAssessment 41
4.18 Test case for PersonalizedLearningPath 42
4.19 Test case for UserInsights 42
4.20 Test case for RAGKnowledgeBase 43

4.21 Test case for AIMentorInteraction 44
4.22 Test case for ProjectAssignment 45
4.23 Test case for UserProjectAssignment 45
4.24 Test case for ProjectSubmission 46

6.1 Test Case TC-AUTH-01: User Login Authentication 64
6.2 Test Case TC-TRACK-01: Learning Track Selection 65
6.3 Test Case TC-ASSESS-01: AI Skill Assessment 66
6.4 Test Case TC-LPATH-01: AI Learning Path Generation 67
6.5 Test Case TC-CONTENT-01: Stage Content Generation and Progress Tracking . . 68
6.6 Test Case TC-RAG-01: RAG Mentor Chat Interaction 69
6.7 Test Case TC-EVAL-01: AI Evaluation Validator Interview 70
6.8 Test Case TC-PROGRESS-01: Progress Insights and Improvement Analysis 71
6.9 Overall Test Metrics Summary 72
6.10 Module-Wise Test Execution Results 72
6.11 Requirements Traceability Matrix 73

Chapter 1 Introduction

Software development has shifted quickly since ChatGPT came out in 2022. Developers now have to pick up new tools and frameworks on a regular basis just to keep their work relevant. Traditional coding courses move too slowly for this kind of pace. They follow fixed schedules and lesson plans that were set months or years ago.

We built this project to close that gap. The platform starts by testing what a learner already knows, then builds a study plan around their gaps and goals. An AI mentor stays with the learner through each lesson and offers feedback as they work. The assessments we include are designed to feel closer to what a junior developer would face on the job. We want learners to understand the concepts well enough to apply them, not just recall them. The rest of this report walks through how we designed the system, how we built it, how we tested it, and what we found.

1.1 Purpose of this Document

This document describes how we designed, built, and tested a learning platform aimed at programmers. We set out to make the learning process faster, more personal, and more focused on problem solving instead of memorization. Most online courses give every learner the same fixed content. Our approach starts with an assessment and then shapes the roadmap around the learner's current level and the career path they want to follow. The question we kept coming back to is whether an AI-driven system can make learning genuinely personal and help developers move through material faster by focusing on how they solve problems. The platform we built tries to answer that. It assesses skills, offers a personalized AI mentor, and reviews code the way a senior developer might. This document covers the research we did, the design decisions we made, how we built the system, what we tested, what worked, what did not, and what we plan to do next.

1.2 Intended Audience

We wrote this report for two groups. The first is the academic side: supervisors, examiners, and faculty who need to assess the project on its technical merit and originality. The second is the practical side: new developers, self-taught coders, and anyone trying to learn programming faster without giving up depth. We kept the technical sections detailed enough for the academic review and wrote the explanations clearly enough that a working programmer can read them and take something useful away.

1.3 Definitions, Acronyms, and Abbreviations

List Of Abbreviations: SDG: Sustainable Development Goal

FYP: Final Year Project

MVP: Minimum Viable Product

UI: User Interface

UX: User Experience

Agile: Agile Development

SCRUM: Scrum Development

REST: Representational State Transfer

ORM: Object-Relational Mapping

CRUD: Create, Read, Update, Delete

API: Application Programming Interface

1.4 Conclusion

GrowWise is our attempt at a more personal way to learn programming. It builds a plan around each learner, keeps an AI mentor on hand, and sets projects that feel closer to real engineering work. The problem we want to solve is that most traditional courses train people to memorize syntax rather than think through problems. This report explains the core idea, the requirements we pulled together, and the system design we arrived at. What we have now is a solid foundation. We still have work left. We plan to keep building on this during FYP-2 until GrowWise reaches a complete, production-ready state.

Chapter 2 Project Vision

This chapter covers the thinking behind the platform. We go over what we think is broken in how programming is taught today, what we want to fix, the goals we set for the project, and the boundaries of what we chose to build. By the end of this chapter, readers should understand why we started this project and how we believe it can change the way programmers learn.

2.1 Problem Domain Overview

We started GrowWise because most programming courses treat every learner the same way. The lessons rarely adjust to a learner's current skill or the way they think. That kind of rigidity slows people down. Learners often end up spending time on material they already understand, and when they do reach gaps in their knowledge, the course is not designed to notice.

The tech industry moves faster than curricula can keep up with. Developers need to learn how to approach problems, not just repeat syntax. Most platforms still measure success by whether a learner can memorize a snippet or follow a tutorial. They rarely push learners to compare approaches or understand why a design choice works.

We want to fix this with AI. The system starts with an assessment, then shapes a personal path around the learner. The point is to avoid wasted time, build actual problem-solving skills, and get the learner closer to what a working developer needs to know.

2.2 Problem Statement

Most learning platforms move slowly and treat every learner identically. They do not consider current skill levels or how individuals approach problems. Developers need a form of teaching that helps them learn to reason through problems rather than memorize tools and frameworks.

2.3 Problem Elaboration

Current learning platforms have several overlapping issues that make it harder for new developers to become job-ready. The main problems we want to address:

1. No personal plan: Most platforms hand the same lesson to every learner, regardless of background or goals. Learners get bored or lose momentum.
2. Wrong starting level: Courses rarely check what a learner already knows. Time gets spent on

familiar material, or learners get thrown into something too advanced.

3. Long and rigid courses: Traditional courses run for weeks or months. By the time they finish, the tools have changed.
4. Focus on memorization: Most courses teach syntax, not reasoning. When a new tool appears, learners are back where they started.
5. Shallow practice: Real learning happens when someone builds something real. Most platforms offer small, isolated exercises instead.

2.4 Goals and Objectives

Our main goal is to build a learning platform that teaches developers how to think through problems, not just how to write code. We want to give learners a faster, more personal experience with AI assistance available whenever they need it.

The specific objectives:

1. Personal study plan: The system begins with an assessment and then creates a plan based on the learner's strengths, weaknesses, and career goals.
2. Right starting point: The learner begins at a level that matches their current skills. No time wasted on the basics if they already know them, and no frustration from jumping straight into advanced material.
3. Flexible pacing: Lessons are short and responsive. If a learner moves quickly, the path adjusts. If they struggle, the path slows down.
4. Reasoning over recall: We focus on teaching engineering thinking, including logic, trade-offs, and system design.
5. Realistic projects: Assignments are closer to what a developer would see at a company. No trivial exercises.
6. Responsive AI mentor: The mentor works like a patient senior engineer. It points to relevant videos or articles, answers questions, and helps when the learner gets stuck.

2.5 Project Scope

This project is a learning platform for developers. It adjusts the lessons for every learner based on their current skill level, learning speed, and career goals such as web development, AI, or security. From there,

it delivers only the content that matters for that path. We wanted the platform to go beyond traditional courses by adding AI mentorship, thorough skill assessments, and a focus on problem-solving methods.

The platform enables users to:

- Register and log in through a secure authentication flow.
- Take an initial test that covers both technical knowledge and problem-solving ability.
- Choose a specialization such as web development, AI, or cybersecurity.
- Receive a custom learning plan based on the test results and the chosen specialization.
- Work with an AI mentor that explains concepts, gives feedback on code, and suggests relevant videos or articles as the learner progresses.
- Complete realistic project assignments that check both problem-solving and system design skills.
- Track their own progress on a personal dashboard that shows strengths, weaknesses, and what to focus on next.

We built the platform on FastAPI for the backend, React.js for the frontend, and Python for the AI components. The AI mentor uses retrieval-augmented generation to pull relevant content and answer learner questions. We followed an Agile workflow with short iterations, frequent testing, early bug fixes, and steady feedback from users.

What the project delivers:

- A working web application that runs in a browser and provides the personalized AI learning experience.
- A full report covering how the system works, the choices behind it, the implementation, and the testing.
- A user guide that walks learners through the platform step by step.

What the project does not include:

- Integration with external learning management systems.
- A mobile application for Android or iOS.
- A large-scale deployment for thousands of concurrent users. A small test server is enough for this stage.

Setting these boundaries early gave us a clear build target and a realistic finish line.

2.6 Sustainable Development Goal (SDG)

GrowWise lines up with UN SDG 4, Quality Education. The goal calls for inclusive, equitable education and lifelong learning for everyone. We support this by using AI to tailor lessons to each learner. Learners who move fast and learners who move slowly get the same attention from the platform. Fast feedback, realistic project work, and current industry topics help users build the skills employers are actually looking for. Because the platform is open to anyone with a browser and can be used on the learner's own schedule, it lowers the barrier for people who cannot afford expensive courses or keep to a fixed class timetable. That is how GrowWise contributes to SDG 4. 2.1.

Figure 2.1: SDG 4 Quality Education, the goal GrowWise aligns with.

2.7 Constraints

Building this platform came with several constraints that shaped our decisions:

- Hardware limits: RAG-based AI and mentor models need significant memory and compute. Our development hardware struggles with heavier workloads and occasionally slows down or crashes.
- Limited time: This is a final year project, so we had only a few months. We could not build out every idea we had or add highly advanced capabilities.
- Integration complexity: The assessment AI, mentor AI, and chat interface need to work together smoothly, and coordinating them has introduced bugs along the way.
- Internet dependency: All AI features and content retrieval require a network connection. Without one, the platform cannot function.

- Small user base for now: This version targets a small test group. A production deployment with thousands of simultaneous users is a later concern.

2.8 Business Opportunity

GrowWise is more than a learning app. It also has a clear commercial angle. Individuals, companies, and schools all have reasons to pay for a platform like this. Traditional courses rarely offer personal pacing or real problem-solving training. GrowWise offers both, and that gives it a distinct position in the market. Many programmers want to gain strong skills quickly, and many companies need to onboard developers fast. Both groups are reasonable customer targets, and both can support a healthy business model.

2.9 Stakeholders Description/ User Characteristics

This section describes who the platform is built for and what they need from it. We spent time understanding both groups so the platform would actually serve them.

2.9.1 Stakeholders Summary

GrowWise is built for two main audiences:

- Developers and junior programmers: This is the primary audience. They range from students and career switchers to self-taught coders and junior developers looking to level up. They want a platform that matches their skill level, adjusts its pace as they learn, and teaches them how to think about problems rather than memorize solutions. The end goal is to get them ready for actual engineering work.
- Tech companies and teams: Larger companies and smaller engineering teams that need to train developers efficiently. They want visibility into who is progressing, scalable training for multiple people at once, and assurance that trainees can handle real projects. Progress reports and certificates are useful to them.

2.9.2 High-Level Goals and Problems of Stakeholders

- Junior programmers: Want a personal learning plan, practical problem-solving experience, and lessons that avoid rote memorization.
- Companies: Want their developers to learn quickly, want clear progress tracking, and want their people to be ready for real work without a long onboarding period.

Understanding what each group needs helps us make design decisions that serve both without compromising either.

2.10 Conclusion

This chapter laid out the core idea behind GrowWise. We covered the problems with traditional programming courses and why they matter, the goals we set for ourselves, and the line between what we planned to build and what we had to leave out given our timeline. We also touched on the commercial case for the platform. With all of that on the table, the next chapter moves into how we designed and built the system.

Chapter 3 Literature Review / Related Work

This chapter covers the tools and platforms we looked at while designing GrowWise. We studied systems that handle personalized learning, project-based practice, or AI-assisted programming instruction. Each one gave us something useful. GrowWise takes what we found most effective from these systems and adds RAG-based knowledge retrieval, AI-driven code review, and a personalized learning plan aimed at junior developers who want to build practical engineering skills.

3.1 Definitions, Acronyms, and Abbreviations

This section clarifies the short forms and specialized terms used in this chapter.

3.1.1 Key Definitions

Adaptive Learning: An approach that adjusts lesson content, pacing, and delivery style based on how the learner progresses.

Agentic AI: AI that can plan and carry out tasks on its own, without needing step-by-step instructions from a user.

Approach-Oriented Learning: Teaching that focuses on how to think about a problem rather than on memorizing specific code.

Learning Path: A sequence of study materials, exercises, and assessments arranged to help a learner reach a goal.

Micro-Learning: Short, focused lessons, usually just a few minutes each, designed to fit into busy schedules.

Personalized Learning: Instruction shaped around an individual learner's pace, interests, and skill level.

RAG (Retrieval-Augmented Generation): An AI technique where the model first searches a knowledge base, then generates an answer grounded in what it found.

Skill Assessment: A test that measures what a learner currently knows and where the gaps are.

3.2 Detailed Literature Review

This section reviews the platforms that inspired GrowWise. We looked at each one carefully and examined how it relates to the work we are doing.

3.2.1 Crio.Do

3.2.1.1 Summary

Crio.Do [1] is an online learning platform designed to bridge the gap between academic programming and actual industry work. It partners with companies such as CRED and assigns learners real projects that match what engineering teams actually build. The focus is on doing, not theorizing. Students and junior developers typically finish with a portfolio of production-style work and a strong base for job applications.

3.2.1.2 Critical Analysis (Strengths and Weaknesses)

Strengths: Offers hands-on experience with projects that mirror industry work. Its partnerships with real companies lend it credibility and align the curriculum with what employers want. Weaknesses: Its tech coverage is limited. Beginners can find the projects too difficult. The time commitment is significant, and working learners often struggle to finish.

3.2.1.3 Relationship to Proposed Work

We took inspiration from Crio.Do's project-based approach. Our platform goes further by using AI to generate personal learning plans for every user, adjusting content as learners progress, and covering a broader set of technologies.

3.2.2 The Odin Project

3.2.2.1 Summary

The Odin Project [2] is a free, open-source curriculum that teaches full-stack web development. It covers HTML, CSS, JavaScript, Git, Node.js, databases, and more, taking learners from beginner to job-ready. Instead of step-by-step video tutorials, it pushes learners to build their own applications, which forces them to develop problem-solving skills along the way. The whole curriculum is free, and a community supports learners along the way. It works well for self-directed people aiming at a web development career.

3.2.2.2 Critical Analysis (Strengths and Weaknesses)

Strengths: Fully free, well-ordered curriculum, and a large community for support. Weaknesses: Heavy self-direction means less motivated learners often drop off. The material is mostly text, which limits interactivity. The exercises do not feel like industry work.

3.2.2.3 Relationship to Proposed Work

Like The Odin Project, we emphasize learning through building. Our platform adds an AI mentor that gives personalized help, adjusts the path based on progress, and delivers fast feedback as learners work.

3.2.3 ProjectLearn.io

3.2.3.1 Summary

ProjectLearn.io [3] is a platform that teaches programming through complete application projects. It offers tutorials for web, mobile, game development, and machine learning. Instead of small exercises, learners build full applications from scratch and see how the parts fit together. Tutorials are grouped by technology and difficulty level, which makes it easy to pick one that matches a learner's current abilities. The platform works well for students who want tangible portfolio pieces rather than pure theory.

3.2.3.2 Critical Analysis (Strengths and Weaknesses)

Strengths: A wide range of projects across several domains. It helps learners build a portfolio and offers content at multiple difficulty levels. Weaknesses: Tutorial quality varies because content comes from many contributors. There is no personalized plan, and the platform does not offer much interactivity beyond the projects themselves.

3.2.3.3 Relationship to Proposed Work

GrowWise shares the project-based philosophy but adds better structure and personalization. Our AI adjusts the path for each learner and delivers faster feedback throughout.

3.2.4 Educative.io

3.2.4.1 Summary

Educative.io [4] targets developers who already know the basics. It covers advanced topics like system design, interview preparation, backend engineering, and topics commonly asked about at larger tech companies. Its strongest feature is the in-browser coding environment, which removes the setup friction. Its AI tracks progress and provides personalized paths along with quick feedback. It works well for mid-level developers preparing for interviews.

3.2.4.2 Critical Analysis (Strengths and Weaknesses)

Strengths: In-browser coding removes setup hassle. Strong catalog for interview and system design topics. AI builds personalized paths. Weaknesses: A subscription is required for most content. Full-stack fundamentals are underrepresented. Absolute beginners may find it too advanced.

3.2.4.3 Relationship to Proposed Work

We borrow the in-browser coding and AI-personalization ideas from Educative.io. Our platform focuses more on beginners and junior developers and offers a full path from the start, with more active AI support along the way.

3.2.5 Codio

3.2.5.1 Summary

Codio [5] is a cloud-based learning platform that offers real coding environments inside the browser. Its courses cover AI, data, security, and full-stack development. The standout feature is that learners use the same tooling as working engineers, including GitHub and CI/CD pipelines. There is no local setup required, which makes it popular with schools and corporate training programs that need to get learners productive quickly.

3.2.5.2 Critical Analysis (Strengths and Weaknesses)

Strengths: Industry-standard tooling with no setup. A varied catalog of courses. Weaknesses: Beginners can feel overwhelmed by the tooling. A subscription is required for serious use. The community around it is smaller, so peer support is limited.

3.2.5.3 Relationship to Proposed Work

Like Codio, we offer a real coding environment. Our platform aims to be more beginner-friendly, with AI-driven personalization, responsive help, and a pace that avoids overwhelming new developers.

3.2.6 Project-Based Learning GitHub Repository

3.2.6.1 Summary

This GitHub repository [6] is a community-maintained collection of project tutorials across many stacks: web, mobile, data science, and a wide range of languages. The list is organized by language, which

makes it easy to browse. Learners build full applications from scratch and end up with portfolio-worthy work. It fits independent learners who do not need a guided curriculum or active mentorship.

3.2.6.2 Critical Analysis (Strengths and Weaknesses)

Strengths: Free, a large catalog of projects across many languages, and useful for portfolio building. Weaknesses: No structured learning path, which makes it easy for beginners to get lost. Tutorial quality varies widely. Everything is text-based, with no integrated coding environment or quick help when learners get stuck.

3.2.6.3 Relationship to Proposed Work

We share the project-based approach but provide structure the repository lacks. Our platform offers a clear step-by-step plan, an AI mentor that is always available, personal pacing, and projects that grow in difficulty as learners improve. This keeps learners from getting stuck or losing direction.

3.2.7 Udemy

3.2.7.1 Summary

Udemy [7] is a large online course marketplace with more than 200,000 courses across programming, web development, data science, and many other fields. Any instructor can publish and sell a course. The content is mostly video-based, with some downloadable files and occasional coding exercises. Courses are inexpensive and can be taken at the learner's own pace. It serves beginners and experienced developers alike, though course quality is inconsistent.

3.2.7.2 Critical Analysis (Strengths and Weaknesses)

Strengths: A huge catalog, low prices, flexible pacing, and ratings that help learners choose. Weaknesses: Courses are static and offer no personalization. Feedback is almost non-existent. Many courses are passive video-watching with limited hands-on work. Completion rates are low because nothing pushes learners to finish.

3.2.7.3 Relationship to Proposed Work

Udemy's strengths lie in catalog size and affordability. Our platform improves on the experience with AI-generated plans, quick feedback, hands-on projects, and a responsive AI mentor. The focus is on thinking and problem-solving, not watching videos and forgetting them later.

3.2.8 LinkedIn Learning

3.2.8.1 Summary

LinkedIn Learning [8] is a video-based learning platform aimed at working professionals. It covers technology, business, and creative skills. The tight LinkedIn integration is its most useful feature: completed courses appear as certificates on a learner's profile, which hiring managers can see directly. Course recommendations are based on the learner's job title and skills. Instructors are typically industry professionals. It suits workers who want to present new skills to employers and progress in their careers.

3.2.8.2 Critical Analysis (Strengths and Weaknesses)

Strengths: Solid video quality, LinkedIn-integrated certificates, career-aligned recommendations, and clear career tracks. Weaknesses: Mostly video-based with little hands-on coding or project work. Personalization is shallow. No rigorous assessments that mirror actual engineering work.

3.2.8.3 Relationship to Proposed Work

We took the career-alignment idea from LinkedIn Learning. Our platform goes further for programmers specifically: an always-available AI mentor, realistic projects, quick feedback, and a learning path that adjusts continuously. The aim is actual capability, not just a certificate on a profile.

3.2.9 Microsoft Learn

3.2.9.1 Summary

Microsoft Learn [9] is a free platform from Microsoft that covers Azure, Microsoft 365, the Power Platform, and other Microsoft products. Learners can work with real cloud tools in a sandbox environment at no cost. It offers clear career tracks, including developer and administrator paths, along with recommendations based on what the learner has completed. Everything is free and includes labs, interactive tutorials, and short assessments. It works well for anyone planning to work with Microsoft technology.

3.2.9.2 Critical Analysis (Strengths and Weaknesses)

Strengths: Free, real Microsoft tooling in sandboxes, well-structured learning paths, and some personalization. Weaknesses: Limited to Microsoft technologies. The exercises are mostly small, not full projects. The assessments measure task completion rather than reasoning or system design.

3.2.9.3 Relationship to Proposed Work

We drew inspiration from Microsoft Learn's personalized recommendations and free lab access. Our platform is not tied to any single vendor and covers a broader set of technologies. The focus is on problem-solving, reasoning, system design, and substantial project work. The AI mentor supports learners at every step with specific, context-aware guidance.

3.2.10 Pluralsight

3.2.10.1 Summary

Pluralsight [10] is a subscription-based learning platform aimed at IT professionals and developers. Its library covers thousands of video courses on programming, cloud, security, and related topics. The Skill IQ assessment is its signature feature. It measures proficiency in a technology quickly and then generates a personal plan that highlights weak areas and suggests courses. Many companies use it because it provides quantifiable skill data that simplifies team training.

3.2.10.2 Critical Analysis

Strengths: Strong assessment system, genuinely personal learning paths, a large catalog suited to enterprise training, and benchmarking against peers. Weaknesses: Mostly video-based with limited hands-on coding. Little emphasis on reasoning, design, or large-scale problem-solving. No quick feedback on learner-written code.

3.2.10.3 Relationship to Proposed Work

Our platform and Pluralsight both begin with an assessment and personalize from there. We go beyond video by offering real projects, reasoning and design evaluation, and an AI mentor that provides specific feedback as learners work. The goal is to develop problem solvers, not course-watchers.

3.2.11 CYPHER Learning (with CYPHER Agent)

3.2.11.1 Summary

CYPHER Learning [11] is an enterprise learning management system used to train large numbers of employees at once. It includes an AI called CYPHER Agent that personalizes content based on job role, current skills, and the employer's needs. The agent identifies gaps and recommends relevant courses. Large organizations use it because it simplifies training rollouts and makes completion easy to track.

3.2.11.2 Critical Analysis

Strengths: The AI personalization works well. It scales to train many people on different topics at once. It saves administrative time. Weaknesses: It is built for general workforce training, not for developer education specifically. It does not teach software reasoning or system design. The content stays general and does not go deep on technical topics.

3.2.11.3 Relationship to Proposed Work

CYPHER demonstrates that AI-assisted personalization works at scale inside an LMS. Our platform applies a similar idea through the RAG mentor but focuses entirely on programmers. The AI delivers senior-developer-style feedback, assesses projects in depth, and teaches learners how to think about problems.

3.2.12 DataCamp

3.2.12.1 Summary

DataCamp [12] is a learning platform specialized in data science, analytics, and AI. It uses short videos followed by in-browser coding exercises, with no local installation required. Learners use Python, R, and SQL in real time. The AI adjusts difficulty based on performance and suggests courses aligned with career goals. It serves everyone from beginners to experienced practitioners in data-related roles.

3.2.12.2 Critical Analysis

Strengths: Real in-browser coding, fast feedback, adaptive AI-driven paths, and strong content for data-related work. Weaknesses: Specializes in data science and limits itself to Python and R, leaving out web development and security. Exercises are small rather than full projects. It does not assess reasoning or system design.

3.2.12.3 Relationship to Proposed Work

Like DataCamp, our platform offers fast feedback and adapts to learners. Our scope is broader, covering more technologies, and we include larger project work that measures reasoning and design the way senior developers would evaluate code.

3.2.13 freeCodeCamp

3.2.13.1 Summary

freeCodeCamp [13] is a nonprofit that offers completely free programming education. The curriculum covers web development, data science, machine learning, and more. Learners build real projects, and some projects contribute to actual nonprofit organizations. Certificates are free on completion and well-regarded by employers. An AI tutor has been added recently. A large community uses the platform because it is free and the content is solid.

3.2.13.2 Critical Analysis

Strengths: Fully free, real projects, recognized certificates, AI tutor, and a large community. Weaknesses: Mostly self-directed, so beginners can lose their way. There is no personalized plan, and nothing actively pushes learners to complete courses. Learners sometimes get stuck for long periods.

3.2.13.3 Relationship to Proposed Work

Like freeCodeCamp, we value real project work and accessible education. Our platform addresses what freeCodeCamp does not: a personalized AI-generated plan, an active AI mentor, fast feedback, and more structure to keep learners moving forward.

3.2.14 Udacity

3.2.14.1 Summary

Udacity [14] offers Nanodegree programs at a premium price. The topics include AI, cloud, data science, and related fields. Content is built in partnership with large tech companies such as Google and Amazon, so it stays current and aligned with industry needs. Projects are substantial and mirror actual engineering work. Human mentors review submissions and provide feedback. Graduates are often well-positioned for senior roles, though the cost is high.

3.2.14.2 Critical Analysis

Strengths: Strong industry-aligned content, substantial projects, human mentors, and deadline structure that encourages completion. Weaknesses: Expensive. Human mentor feedback can be slow. The learning path is largely fixed and does not adjust dynamically to learner performance.

3.2.14.3 Relationship to Proposed Work

Like Udacity, we focus on realistic projects and job readiness. Our approach replaces slow human mentor feedback with an always-available AI mentor backed by RAG, and the path adjusts as learners progress. Learners pay less and get a more personalized experience.

3.2.15 CodeCombat

3.2.15.1 Summary

CodeCombat [15] teaches programming through a fantasy game. Learners write actual Python or JavaScript to control characters and solve challenges. It works especially well for children and absolute beginners. When learners get stuck, the AI offers hints. The difficulty increases gradually as the game progresses. It makes learning the basics of code enjoyable rather than tedious.

3.2.15.2 Critical Analysis

Strengths: Engaging and game-like, which keeps learners coming back. Instant feedback and AI-driven hints keep them unstuck. A strong fit for younger learners or first-time coders. Weaknesses: The game format does not transfer directly to real project work or system design. Problems have single correct answers, which is not how engineering work operates. Not well suited to intermediate or advanced developers.

3.2.15.3 Relationship to Proposed Work

Like CodeCombat, our platform uses AI to provide fast guidance and keep learners from getting stuck. Our focus is on working developers rather than gamified learning for beginners. We assign realistic projects and assess reasoning and design the way a senior engineer would. The goal is job readiness, not gameplay.

3.3 Literature Review Summary Table

The following table summarizes the platforms we reviewed, including their key features, their relevance to GrowWise, and the limitations we identified.

Table 3.1: Summary of Learning Platforms Reviewed

Platform	Key Features	Relevance to GrowWise	Identified Limitations
Crio.Do [1]	Project-based learning with real-world company projects	Provides insight into hands-on skill development and project experience	Limited to certain tech stacks; time-intensive; may overwhelm beginners
The Odin Project [2]	Free, open-source full-stack web development curriculum	Emphasizes learning by doing and portfolio building	Self-paced nature may reduce accountability; text-heavy; limited industry exposure
ProjectLearn.io [3]	Curated project tutorials across multiple domains (web, mobile, ML, game dev)	Focus on project-based learning and portfolio creation	Quality varies across creators; lacks personalized paths; limited interactive features
Educative.io [4]	Interactive coding courses with AI personalization	Provides hands-on coding practice and adaptive learning paths	Subscription required for some courses; limited full-stack coverage; better for intermediate learners
Codio [5]	Real-time coding environments with GitHub	Offers immersive, practical coding experience and workflow integration	Can be complex for beginners; subscription required; limited community support
Project-Based Learning GitHub Repository [6]	Curated tutorials for multiple languages and frameworks	Encourages learning by doing and project portfolios	Lacks structured learning paths; quality varies; limited interactivity
Udemy [7]	Massive course catalog across various technologies	Wide accessibility; self-paced learning	Static content; limited personalization; minimal real-time feedback
LinkedIn Learning [8]	Professional courses with LinkedIn integration	Structured career-oriented learning paths	Mostly video-based; minimal coding practice; limited project validation
Microsoft Learn [9]	Personalized learning paths with hands-on labs	Interactive exercises and adaptive recommendations	Focused on Microsoft technologies; limited assessment of problem-solving or architecture skills
Pluralsight [10]	Skill IQ assessment, adaptive testing, personalized learning paths, enterprise training	Assessment-first approach, skill-level matching, industry benchmarking	Video-based delivery, limited interactive practice, focus on skill acquisition over architectural evaluation
CYPHER Learning [11]	Agentic AI integration, adaptive learning, enterprise LMS, AI skills management	Personal learning agent validation, agentic AI philosophy, adaptive learning capabilities	Enterprise focus, generalized content, limited specialized developer training
DataCamp [12]	Interactive coding exercises, micro-assessments, AI-recommended paths, real-time adaptation	Interactive learning model, continuous assessment, adaptive pacing	Domain-specific focus, limited architectural reasoning, emphasis on code mastery over problem-solving approach
freeCodeCamp [13]	Free curriculum, real-world projects, community-driven learning, AI tutoring integration	Project-based learning, accessibility, real-world application	Self-paced nature, limited personalized guidance, overwhelming for beginners, lack of accountability
Udacity [14]	Nanodegree programs, industry partnerships, project-based learning, human mentor support	Industry-ready focus, project-intensive learning, career alignment	High cost, human-gated feedback, limited scalability, fixed learning paths
CodeCombat [15]	Gamified learning, interactive coding, AI-driven hints, immersive environments	Engagement strategies, interactive environments, AI-driven guidance	Gamification limitations, simplified complexity, limited real-world architectural assessment

3.4 Conclusion

In this chapter we examined a range of learning platforms: Crio.Do, The Odin Project, ProjectLearn.io, Educative.io, Codio, the GitHub project-based learning repository, Udemy, LinkedIn Learning, Microsoft Learn, Pluralsight, CYPHER Learning, DataCamp, freeCodeCamp, Udacity, and CodeCombat. These platforms emphasize learning through projects, hands-on practice, and coding exercises, which build solid skills and portfolios. Most do not offer genuinely adaptive learning that changes with the learner, AI mentors, instant feedback on conceptual understanding, or personal paths tied to current skill and career goals.

This review exposed gaps that GrowWise can fill. Our platform combines AI-driven personalization, intelligent resource retrieval, and project-based assessment to teach learners how to reason through problems. What we learned from these platforms guided the design of GrowWise. We addressed their weaknesses while strengthening learners' critical thinking, problem-solving, and practical programming skills.

Chapter 4 Software Requirement Specifications

This chapter describes the main features of the project. It covers what the system should do, how it should work, the use cases it supports, and the risks we have to manage.

4.1 List of Features

The main features of GrowWise:

- Secure sign-up and login with straightforward profile creation.
- Choice of learning track, such as AI, web development, or game development.
- An initial skill assessment on day one that identifies strengths and weaknesses.
- A personalized study plan generated from the assessment results and the chosen track.
- An AI mentor available during every lesson that explains concepts, offers hints, and suggests relevant videos or articles.
- AI-assigned realistic projects, followed by an AI-driven code review that evaluates the code, the reasoning behind it, and the design. The feedback resembles what a senior developer would give.
- A progress dashboard that highlights strengths, remaining gaps, and suggested next steps.
- Admin tools for adding new tracks, expanding the AI's knowledge base, and fixing issues when they arise.

4.2 Functional Requirements

This section lists what the system can do for each user role.

4.2.1 Functional Requirements of Programmers / Developers

Programmer users can:

- Create an account and log in.
- Choose a track such as AI, web development, or game development.
- Take the initial skill assessment so the system can gauge their level.
- Receive a personal study plan.
- Work through each lesson alongside the AI mentor, with examples and video references available.

- Complete short exercises or mini-projects in every lesson.
- See progress over time along with feedback on strengths and weaknesses.
- Complete a final capstone project that the AI evaluates for both code and reasoning.

4.2.2 Functional Requirements of Admin

Admin users can:

- Create new tracks or update existing ones.
- Add lessons, videos, and articles.
- Upload new documents into the RAG knowledge base to improve AI responses.
- Review user learning paths and fix issues when something goes wrong.

4.2.3 Functional Requirements of System

The system itself can:

- Keep user accounts secure.
- Build a personalized plan for each new user.
- Provide AI mentor support during every lesson.
- Track progress and return feedback quickly.
- Assign major projects and evaluate submitted code with AI that mirrors senior-level review.
- Maintain the knowledge base so the AI has current material to draw from.

4.3 Quality Attributes

GrowWise is designed to be reliable and easy to use. The main qualities we focused on:

- Reliability: The AI mentor and evaluator return accurate responses. The platform does not crash under normal load.
- Usability: Screens are simple and clear. Lessons, videos, and feedback are easy to find.
- Maintainability: The code is organized into clean, modular sections so future features and fixes are straightforward.
- Performance: The platform stays responsive even when several users are active at the same time.

- Flexibility: New tracks, knowledge sources, and features can be added without major changes.

4.4 Non-Functional Requirements

This section covers the non-functional requirements of the GrowWise platform, including performance, reliability, usability, reusability, and extensibility.

4.4.1 Performance

The system should help users learn efficiently and keep overall learning time short.

4.4.2 Reliability

The platform should be available continuously so learners can access their paths and the AI mentor without interruption.

4.4.3 Usability

The platform should be simple to use. Users should quickly understand how to navigate and follow their learning path.

4.4.4 Reusability

The code should be modular so components can be reused for new features or updates.

4.4.5 Extensibility

New learning paths, AI guidance, and project types should be easy to add over time.

4.5 Assumptions

We based the design on the following assumptions:

- Users have a web browser on their computer or device that supports JavaScript.
- Users have an internet connection stable enough to reach the platform.
- Users know basic web navigation, such as clicking links and filling in forms.
- Users have some familiarity with programming, enough to follow the learning paths.
- Admins know how to create learning paths and manage the AI-driven knowledge base.

4.6 Use Cases

This section lists the use cases our application supports. The tables below document each one in detail.

4.6.1 User Login

Table 4.1: Use Case: User Login

Name	User Login		
Actors	Programmer, Developer		
Summary	User enters email and password, and after verification, is redirected to the dashboard.		
Pre-Conditions	User must have an account and must not be already logged in.		
Post-Conditions	User session is established, and dashboard is accessible.		
Special Reqs	None		
Basic Flow			
Actor Action		System Response	
1	User opens login page	2	System displays email and password fields
2	User enters valid credentials	4	System verifies, creates session, redirects to dashboard
Alternative Flow			
3	User enters invalid credentials	4-A	System displays “Incorrect email or password” message

4.6.2 Track Selection

Table 4.2: Use Case: Track Selection

Name	Track Selection		
Actors	Programmer, Developer		
Summary	User selects a learning track to start personalized learning.		
Pre-Conditions	User must be logged in.		
Post-Conditions	Selected track is saved for learning path generation.		
Special Reqs	None		
Basic Flow			
Actor Action		System Response	
1	User opens track selection page	2	System displays available tracks
2	User selects a track	4	System saves selection and confirms
Alternative Flow			
3	User does not select a track	4-A	System prompts: “Please select a track to continue”

4.6.3 Skill Assessment

Table 4.3: Use Case: Skill Assessment

Name	Skill Assessment		
Actors	Programmer, Developer		
Summary	User takes a test to check their current skill level.		
Pre-Conditions	User must select a track.		
Post-Conditions	Skill level is saved in the system.		
Special Reqs	Assessment covers technical and approach-based skills.		
Basic Flow			
Actor Action		System Response	
1	User starts assessment	2	System displays questions
2	User answers questions	4	System evaluates answers and calculates score
Alternative Flow			
3	User leaves assessment incomplete	4-A	System saves progress for later completion

4.6.4 AI Path Generation

Table 4.4: Use Case: AI Path Generation

Name	AI Path Generation		
Actors	Programmer, Developer		
Summary	System generates a personalized learning path based on assessment results.		
Pre-Conditions	Skill assessment must be completed.		
Post-Conditions	Personalized path is available for user.		
Special Reqs	Path should focus on approach-oriented learning.		
Basic Flow			
Actor Action		System Response	
1	System analyzes user score	2	Generates adaptive learning path
2	System displays path	4	User can start modules
Alternative Flow			
3	System cannot generate path	4-A	System shows message: “Try assessment again or contact support”

4.6.5 Agentic RAG Assistance

Table 4.5: Use Case: RAG AI Assistance

Name	RAG AI Assistance		
Actors	Programmer, Developer		
Summary	Users ask questions to AI assistant at any step for help.		
Pre-Conditions	User must be following a module.		
Post-Conditions	User receives guidance or answers from AI.		
Special Reqs	AI must provide context-aware answers.		
Basic Flow			
Actor Action		System Response	
1	User asks a question	2	System retrieves answer from knowledge base
2	User reads answer	4	System logs interaction for feedback
Alternative Flow			
3	AI cannot answer	4-A	System suggests reference material or next steps

4.6.6 Project-Based Assignment

Table 4.6: Use Case: Project Assignment

Name	Project Assignment		
Actors	Programmer, Developer		
Summary	AI assigns a project after completing modules, simulating a senior engineer.		
Pre-Conditions	Required modules must be completed.		
Post-Conditions	User submits project, system evaluates approach and understanding.		
Special Reqs	Evaluation focuses on problem-solving and approach.		
Basic Flow			
Actor Action		System Response	
1	System assigns project	2	User receives instructions
2	User submits project	4	System evaluates and provides feedback
Alternative Flow			
3	User cannot complete project	4-A	System allows resubmission with guidance

4.6.7 Admin Login

Table 4.7: Use Case: Admin Login

Name	Admin Login		
Actors	Admin		
Summary	Admin logs in to manage learning paths and RAG knowledge base.		
Pre-Conditions	Admin account exists.		
Post-Conditions	Admin can access dashboard.		
Special Reqs	None		
Basic Flow			
Actor Action		System Response	
1	Admin opens login page	2	System shows email and password fields
2	Admin enters valid credentials	4	System grants dashboard access
Alternative Flow			
3	Admin enters invalid credentials	4-A	System shows error message

4.6.8 Manage Learning Paths

Table 4.8: Use Case: Learning Path Management

Name	Learning Path Management		
Actors	Admin		
Summary	Admin creates or updates learning paths for all tracks.		
Pre-Conditions	Admin must be logged in.		
Post-Conditions	Updated paths are available for users.		
Special Reqs	Paths must follow approach-oriented methodology.		
Basic Flow			
Actor Action		System Response	
1	Admin opens path management	2	System displays existing paths
2	Admin adds/edits path	4	System saves updates
Alternative Flow			
3	Admin enters invalid info	4-A	System prompts for correction

4.6.9 Manage RAG Knowledge Base

Table 4.9: Use Case: RAG Knowledge Management

Name	RAG Knowledge Management		
Actors	Admin		
Summary	Admin adds or updates resources for AI assistant.		
Pre-Conditions	Admin must be logged in.		
Post-Conditions	AI knowledge base is updated and ready for queries.		
Special Reqs	None		
Basic Flow			
Actor Action		System Response	
1	Admin opens RAG management	2	System displays current resources
2	Admin adds/updates content	4	System updates knowledge base
Alternative Flow			
3	Content fails to save	4-A	System shows error message

4.6.10 Monitor User Progress

Table 4.10: Use Case: User Progress Monitoring

Name	User Progress Monitoring		
Actors	Admin		
Summary	Admin views user progress and completion of modules and projects.		
Pre-Conditions	Admin must be logged in.		
Post-Conditions	Admin sees real-time analytics.		
Special Reqs	Data must be accurate and updated.		
Basic Flow			
Actor Action		System Response	
1	Admin opens analytics page	2	System displays progress charts
2	Admin filters by track/user	4	System updates view
Alternative Flow			
3	Data fails to load	4-A	System shows message: “Try again later”

4.6.11 Learning Path Module Access

Table 4.11: Use Case: Module Access

Name	Module Access		
Actors	Programmer, Developer		
Summary	User follows the AI-generated path, accessing modules step by step.		
Pre-Conditions	User must have a generated path.		
Post-Conditions	Progress is tracked, and modules are completed.		
Special Reqs	Modules should be interactive and practical.		
Basic Flow			
Actor Action		System Response	
1	User opens module	2	System displays content
2	User completes module	4	System marks completion and unlocks next module
Alternative Flow			
3	User skips module	4-A	System allows revisiting later and reminds user

4.7 Hardware and Software Requirements

This section lists the hardware and software needed to develop and deploy the platform.

4.7.1 Hardware Requirements

The platform needs the following hardware to run reliably:

- Computer: A standard desktop or laptop that can run a modern web browser.
- Internet connection: A stable connection is required to reach the platform and use the AI features.
- AWS Virtual Private Server (VPS): The backend runs on an AWS VPS to provide scalability, performance, and secure data handling.

4.7.2 Software Requirements

The following software components are used:

- FastAPI

- Next.js
- OpenAI or Gemini API
- Supabase (PostgreSQL)
- AWS (Virtual Private Server)
- Docker
- Git and GitHub/GitLab
- Python 3.10+
- Node.js 18+
- VS Code
- Postman

4.8 Graphical User Interface

This section shows the graphical interface of GrowWise. Each figure walks through a part of the user experience, from login and path selection to assessments, AI evaluation, and progress analytics.

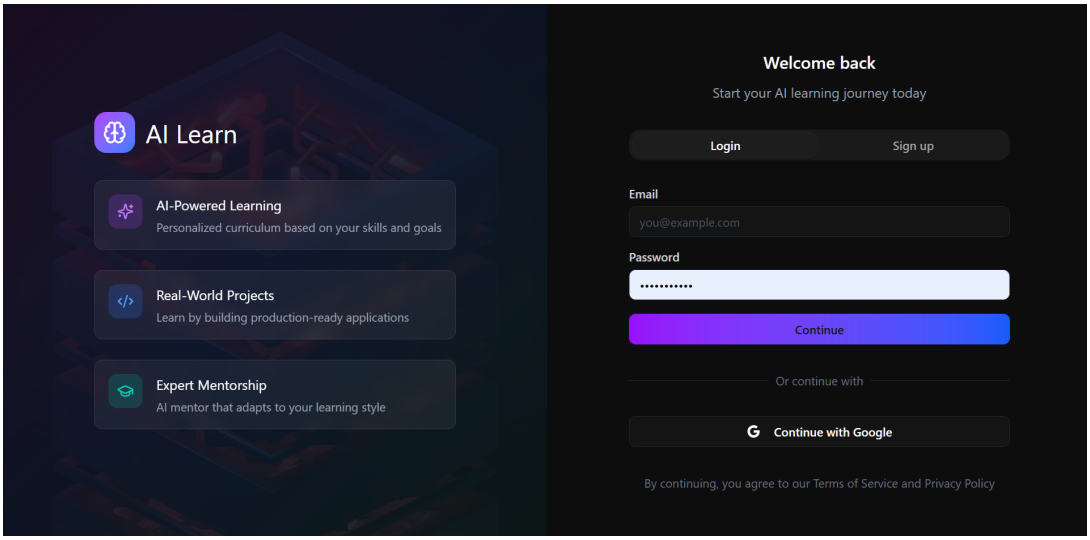

Figure 4.1: Login screen for users to enter credentials and access the system.

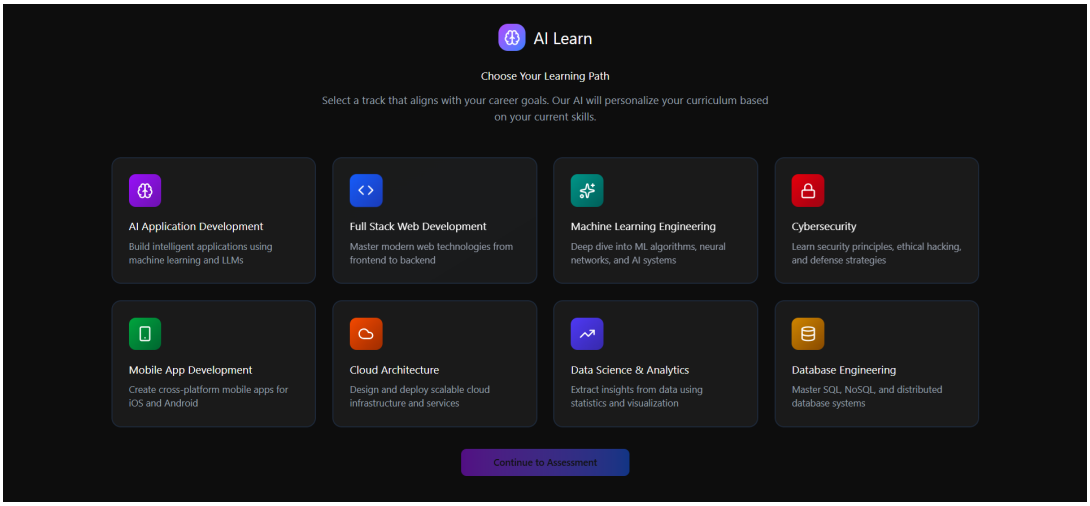

Figure 4.2: Track selection screen where users can choose their learning path.

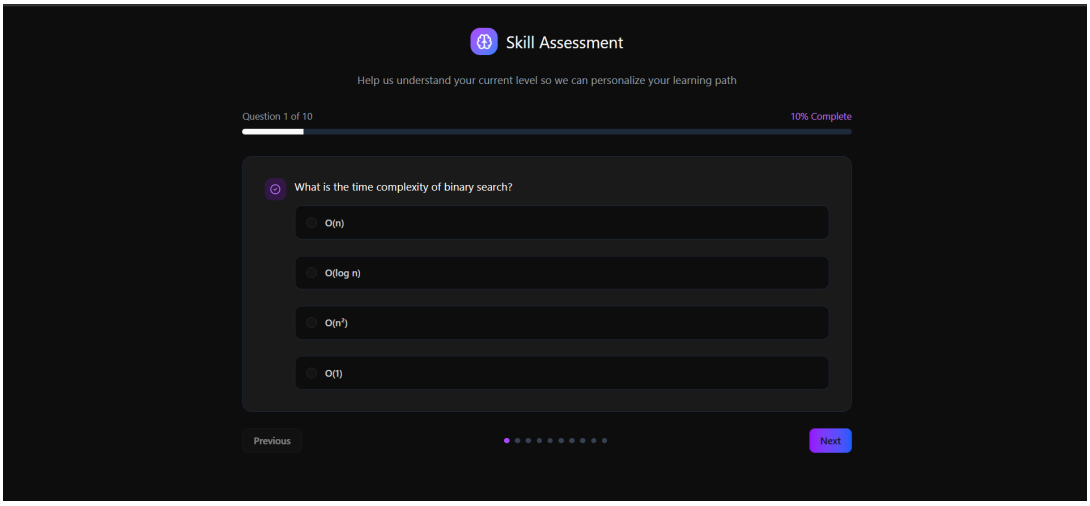

Figure 4.3: Skill assessment page for evaluating the user's technical and problem-solving abilities.

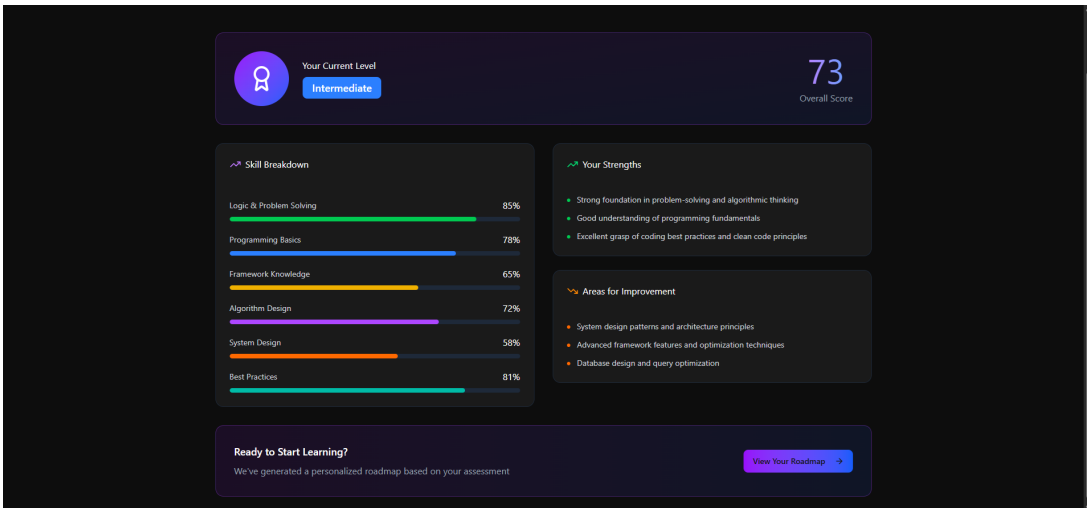

Figure 4.4: Assessment report showing the user's skill level and performance summary.

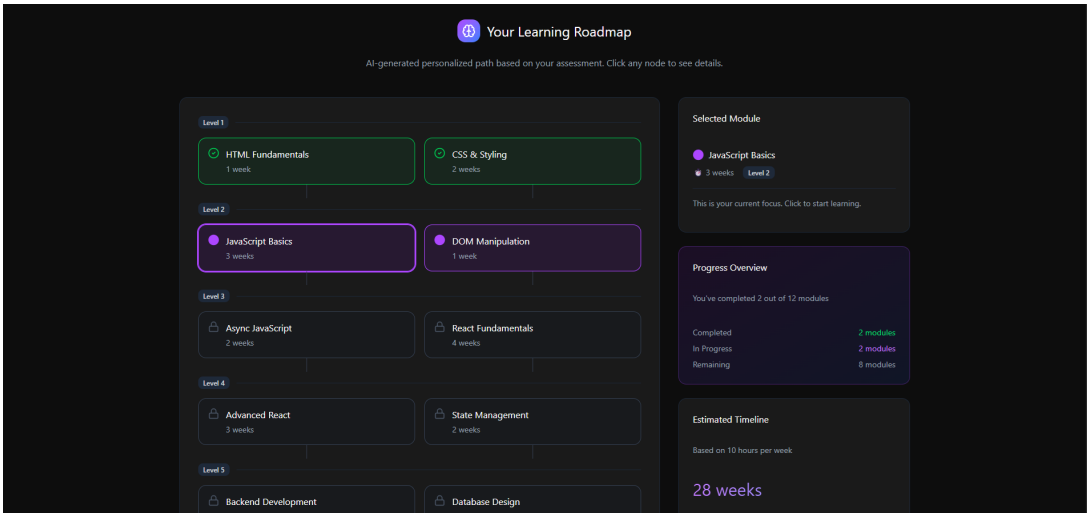

Figure 4.5: Personalized roadmap showing the AI-generated learning journey and module sequence.

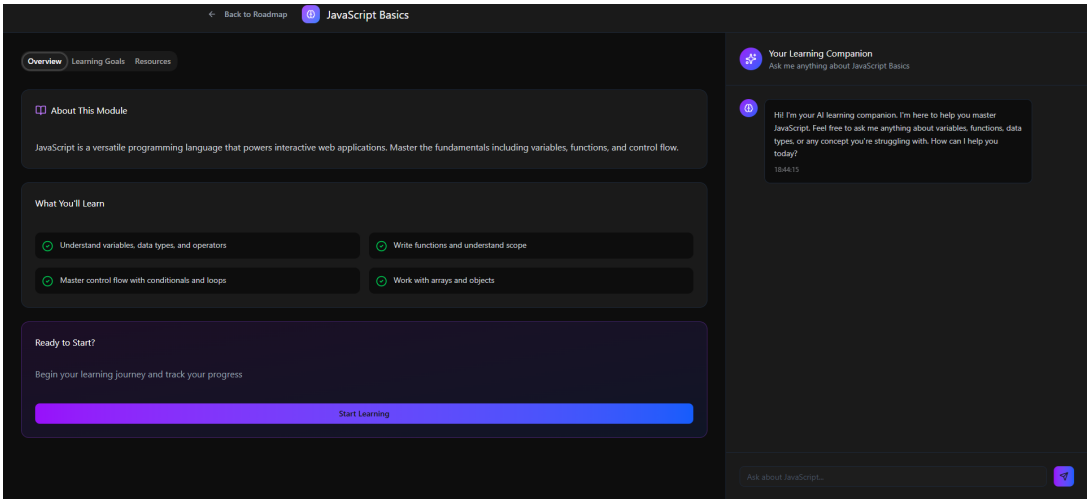

Figure 4.6: Learning module interface with study content and an integrated AI chatbot.

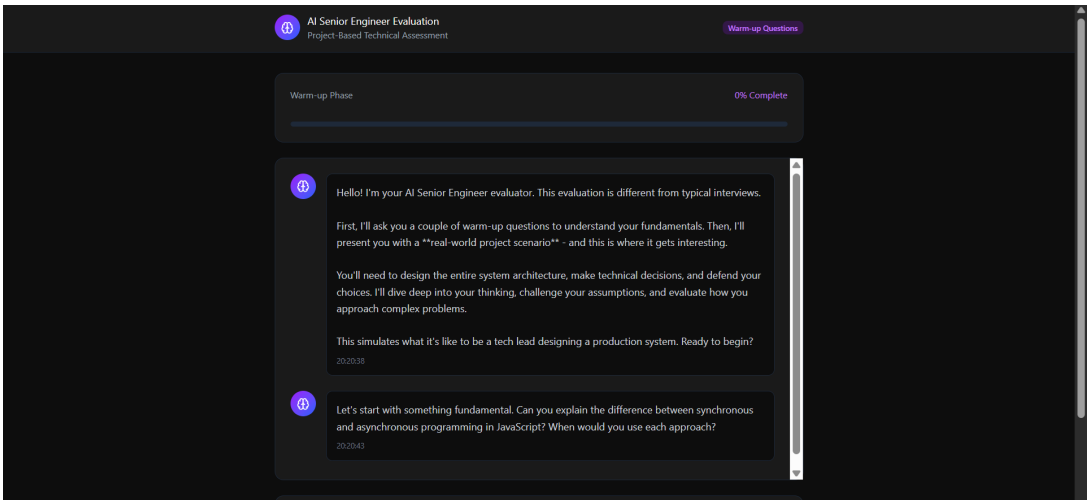

Figure 4.7: AI software engineer screen that assigns projects and evaluates user responses.

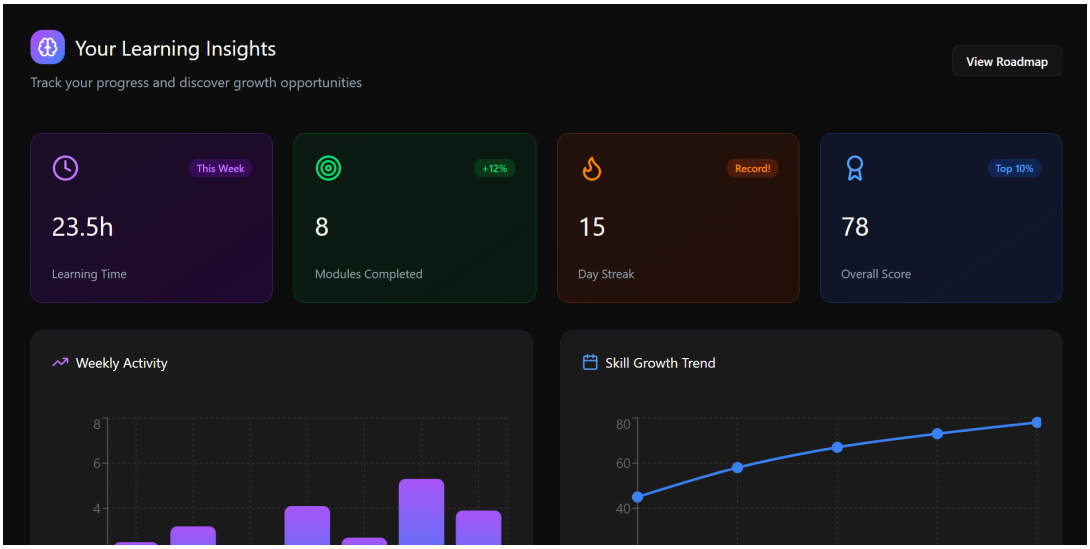

Figure 4.8: Insights dashboard with user progress analytics.

Figure 4.9: Advanced insights with AI evaluation and performance graphs.

4.9 Database Design (if required; this means if you are using noSQL, you will not provide ER but the other design element and data dictonary should still be there. Explain and elaborate your DB design)

4.9.1 ER Diagram

GrowWise runs on PostgreSQL through Supabase as the main relational database. We use the pgvector extension to store and retrieve the embeddings the RAG mentor depends on. The schema follows a normalized relational model organized into four functional modules: user management and authentica-

tion, the learning track system, progress and assessment with AI services, and the project workflow and evaluation pipeline.

The entity-relationship diagram below shows the complete schema, including all entities, their attributes, primary and foreign keys, and cardinality constraints. The design keeps data consistent through normalization, supports concurrent users, and can handle the complex queries needed for generating personal learning paths and AI-driven insights.

We structured the schema to support three complementary PlantUML views. View 1 covers the user journey and progress tracking. View 2 focuses on AI services and knowledge management. View 3 covers project workflow and evaluation. The PlantUML files (growwise_view1_user_journey.puml, growwise_view2_ai_services.puml, and growwise_view3_projects.puml) give visual representations of the database from each of these perspectives.

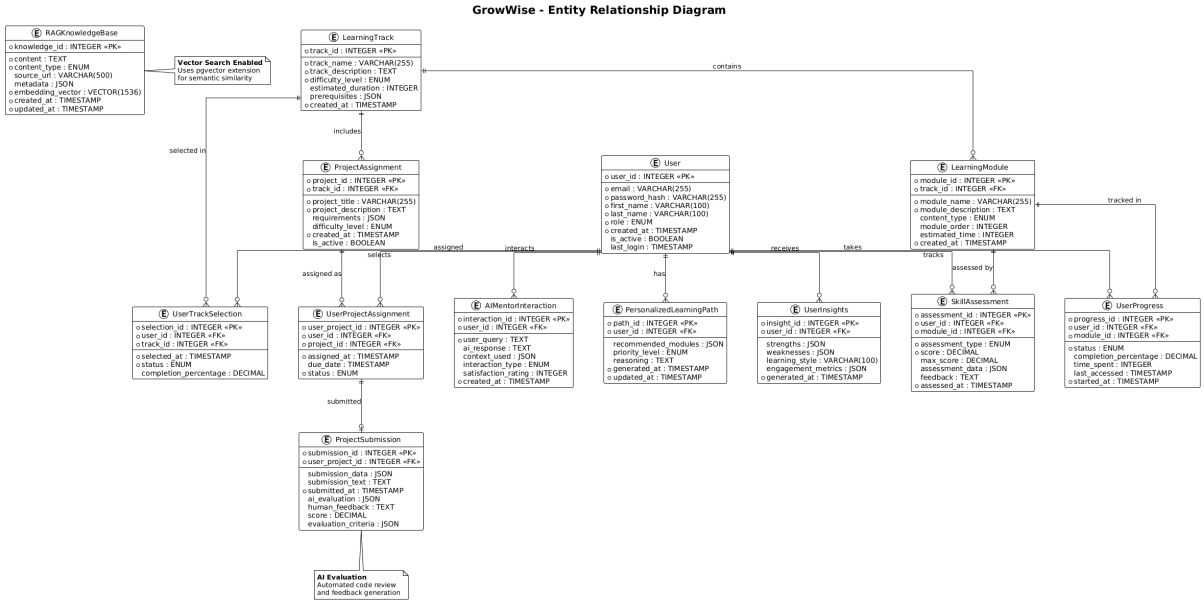

Figure 4.10: GrowWise Entity-Relationship Diagram with all entities, keys, and relationships.

Key database entities and relationships:

- User management: The *User* entity is the central anchor. It has one-to-many relationships with UserTrackSelection, UserProgress, SkillAssessment, PersonalizedLearningPath, UserInsights, AIMentorInteraction, and UserProjectAssignment.
- Learning content structure: The *LearningTrack* entity has a composition relationship with *LearningModule* (one-to-many). Each track contains multiple ordered modules. Tracks also relate to UserTrackSelection, SkillAssessment, PersonalizedLearningPath, and ProjectAssignment.
- Progress tracking: *UserProgress* connects users to specific modules and records completion status

and progress percentages. *SkillAssessment* ties users and tracks together to measure proficiency, which then feeds the generation of personalized learning paths.

- AI services: *RAGKnowledgeBase* stores module-specific content along with vector embeddings for semantic search. It supports the *AIMentorInteraction* entity, which logs every Q&A exchange with the AI mentor. *PersonalizedLearningPath* holds the AI-generated module sequences built from assessment results.
- Project management: *ProjectAssignment* defines reusable project templates that tie to tracks. *UserProjectAssignment* creates individual project instances for each user. *ProjectSubmission* stores the submitted work along with both AI and human evaluation data.

The complete ER diagram, showing every entity, attribute, primary key, foreign key, and cardinality, is rendered through the three PlantUML diagrams referenced above. Together they give a full picture of the GrowWise database.

4.9.2 Data Dictionary

The data dictionary documents every table, its columns, data types, constraints, and purpose. This section captures the full schema for the GrowWise platform, grouped by functional module for easier navigation and maintenance. Each entry lists the column name, its data type, applicable constraints such as primary keys, foreign keys, uniqueness, and null rules, along with a short description of what the column represents.

Table 4.12: Test case for User

Attribute Name	Data Type	Constraints	Description
user_id	INTEGER	PK, AUTO INCREMENT, NOT NULL	Unique identifier for each user
email	VARCHAR(255)	UNIQUE, NOT NULL	User's email address for login and communication
password_hash	VARCHAR(255)	NOT NULL	Hashed password for secure authentication
first_name	VARCHAR(100)	NOT NULL	User's first name
last_name	VARCHAR(100)	NOT NULL	User's last name
user_type	ENUM('learner', 'admin')	NOT NULL, DEFAULT 'learner'	Role-based user classification
created_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Account creation timestamp
is_active	BOOLEAN	NOT NULL, DEFAULT TRUE	Account status flag

Table 4.13: Test case for LearningTrack

Attribute Name	Data Type	Constraints	Description
track_id	INTEGER	PK, AUTO INCREMENT, NOT NULL	Unique identifier for each learning track
track_name	VARCHAR(200)	NOT NULL	Name of the learning track (e.g., "Web Development", "AI/ML")
description	TEXT	NULL	Detailed description of track goals and curriculum
difficulty_level	ENUM('beginner', 'intermediate', 'advanced')	NOT NULL	Overall difficulty rating of the track
estimated_duration_hours	INTEGER	NOT NULL	Approximate time required to complete the track

Table 4.14: Test case for LearningModule

Attribute Name	Data Type	Constraints	Description
module_id	INTEGER	PK, AUTO INCREMENT, NOT NULL	Unique identifier for each module
track_id	INTEGER	FK(LearningTrack), NOT NULL	Reference to parent learning track
module_name	VARCHAR(200)	NOT NULL	Name of the learning module
module_order	INTEGER	NOT NULL	Sequence position within the track
content_type	ENUM('video', 'text', 'interactive', 'quiz')	NOT NULL	Type of learning content
estimated_duration_minutes	INTEGER	NOT NULL	Expected completion time for this module
learning_objectives	TEXT	NULL	Specific learning goals for this module

Table 4.15: Test case for UserTrackSelection

Attribute Name	Data Type	Constraints	Description
selection_id	INTEGER	PK, AUTO INCREMENT, NOT NULL	Unique identifier for each selection
user_id	INTEGER	FK(User), NOT NULL	Reference to the user making the selection
track_id	INTEGER	FK(LearningTrack), NOT NULL	Reference to the selected track
selected_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	When the track was selected
is_active	BOOLEAN	NOT NULL, DEFAULT TRUE	Whether this selection is currently active

Table 4.16: Test case for UserProgress

Attribute Name	Data Type	Constraints	Description
progress_id	INTEGER	PK, AUTO INCREMENT, NOT NULL	Unique identifier for each progress record
user_id	INTEGER	FK(User), NOT NULL	Reference to the user
module_id	INTEGER	FK(LearningModule), NOT NULL	Reference to the learning module
status	ENUM('not_started', 'in_progress', 'completed')	NOT NULL, DEFAULT 'not_started'	Current completion status
started_at	TIMESTAMP	NULL	When the user began this module
completed_at	TIMESTAMP	NULL	When the user completed this module
progress_percentage	DECIMAL(5,2)	NOT NULL, DEFAULT 0.00	Percentage of module completed (0-100)

Table 4.17: Test case for SkillAssessment

Attribute Name	Data Type	Constraints	Description
assessment_id	INTEGER	PK, AUTO INCREMENT, NOT NULL	Unique identifier for each assessment
user_id	INTEGER	FK(User), NOT NULL	Reference to the assessed user
track_id	INTEGER	FK(LearningTrack), NOT NULL	Track being assessed
total_questions	INTEGER	NOT NULL	Total number of assessment questions
score_percentage	DECIMAL(5,2)	NOT NULL	User's score (0-100)
assessment_data	JSON	NULL	Detailed question-by-question results
completed_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Assessment completion time

Table 4.18: Test case for PersonalizedLearningPath

Attribute Name	Data Type	Constraints	Description
path_id	INTEGER	PK, AUTO INCREMENT, NOT NULL	Unique identifier for each learning path
user_id	INTEGER	FK(User), NOT NULL	Reference to the user
track_id	INTEGER	FK(LearningTrack), NOT NULL	Reference to the learning track
assessment_id	INTEGER	FK(SkillAssessment), NULL	Assessment that generated this path
path_data	JSON	NOT NULL	AI-generated sequence of modules and milestones
status	ENUM('active', 'completed', 'archived')	NOT NULL, DEFAULT 'active'	Current path status
created_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Path generation timestamp

Table 4.19: Test case for UserInsights

Attribute Name	Data Type	Constraints	Description
insight_id	INTEGER	PK, AUTO INCREMENT, NOT NULL	Unique identifier for each insight
user_id	INTEGER	FK(User), NOT NULL	Reference to the user
insight_type	ENUM('strength', 'weakness', 'recommendation', 'milestone')	NOT NULL	Category of insight
insight_data	JSON	NOT NULL	Detailed insight content and metrics
priority	ENUM('low', 'medium', 'high')	NOT NULL, DEFAULT 'medium'	Importance level of the insight
created_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Insight generation timestamp

Table 4.20: Test case for RAGKnowledgeBase

Attribute Name	Data Type	Constraints	Description
knowledge_id	INTEGER	PK, AUTO INCREMENT, NOT NULL	Unique identifier for each knowledge entry
module_id	INTEGER	FK(LearningModule), NULL	Associated learning module
content_type	ENUM('documentation', 'example', 'explanation', 'reference')	NOT NULL	Type of knowledge content
title	VARCHAR(300)	NOT NULL	Title or heading of the knowledge entry
content	TEXT	NOT NULL	Full text content for retrieval
metadata	JSON	NULL	Additional structured information
embedding_vector	VECTOR(1536)	NOT NULL	Vector embedding for semantic search (pgvector)
is_active	BOOLEAN	NOT NULL, DEFAULT TRUE	Whether this entry is currently searchable
created_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Entry creation timestamp

Table 4.21: Test case for AIMentorInteraction

Attribute Name	Data Type	Constraints	Description
interaction_id	INTEGER	PK, AUTO INCREMENT, NOT NULL	Unique identifier for each interaction
user_id	INTEGER	FK(User), NOT NULL	Reference to the user asking the question
module_id	INTEGER	FK(LearningModule), NULL	Context module if applicable
user_question	TEXT	NOT NULL	User's original question
ai_response	TEXT	NOT NULL	AI-generated response
context_data	JSON	NULL	Retrieved RAG context and sources
response_time_ms	INTEGER	NULL	Response generation time in milliseconds
feedback_rating	INTEGER	NULL, CHECK (feedback_rating BETWEEN 1 AND 5)	User satisfaction rating (1-5)
created_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Interaction timestamp

Table 4.22: Test case for ProjectAssignment

Attribute Name	Data Type	Constraints	Description
project_id	INTEGER	PK, AUTO INCREMENT, NOT NULL	Unique identifier for each project template
track_id	INTEGER	FK(LearningTrack), NOT NULL	Associated learning track
project_title	VARCHAR(300)	NOT NULL	Name of the project
project_description	TEXT	NOT NULL	Detailed project requirements and objectives
requirements	JSON	NOT NULL	Technical and functional requirements
difficulty_level	ENUM('beginner', 'intermediate', 'advanced')	NOT NULL	Project complexity level
created_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Project creation timestamp
is_active	BOOLEAN	NOT NULL, DEFAULT TRUE	Whether project is currently assignable

Table 4.23: Test case for UserProjectAssignment

Attribute Name	Data Type	Constraints	Description
user_project_id	INTEGER	PK, AUTO INCREMENT, NOT NULL	Unique identifier for each user assignment
user_id	INTEGER	FK(User), NOT NULL	Reference to the assigned user
project_id	INTEGER	FK(ProjectAssignment), NOT NULL	Reference to the project template
assigned_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Assignment timestamp
due_date	TIMESTAMP	NULL	Project deadline
status	ENUM('assigned', 'in_progress', 'submitted', 'evaluated')	NOT NULL, DEFAULT 'assigned'	Current project status

Table 4.24: Test case for ProjectSubmission

Attribute Name	Data Type	Constraints	Description
submission_id	INTEGER	PK, AUTO INCREMENT, NOT NULL	Unique identifier for each submission
user_project_id	INTEGER	FK(UserProjectAssignment), NOT NULL	Reference to the user's project assignment
submission_data	JSON	NOT NULL	Project files, links, and deliverables
submission_text	TEXT	NULL	User's written explanation or documentation
submitted_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Submission timestamp
ai_evaluation	JSON	NULL	AI-generated evaluation results and feedback
human_feedback	TEXT	NULL	Optional instructor or admin feedback
score	DECIMAL(5,2)	NULL	Final project score (0-100)
evaluation_criteria	JSON	NULL	Rubric and evaluation breakdown

4.10 Risk Analysis

This section covers potential project risks and how we plan to address them.

4.10.1 Technical Risks

The platform can run into bugs, server outages, or incorrect AI responses. The OpenAI and Gemini APIs can also change without much warning or go down entirely. We mitigate this through thorough testing and a fallback plan that can switch providers if needed.

4.10.2 Business Risks

The online learning space is crowded, so gaining users is difficult. API and infrastructure costs can climb quickly. We believe the platform's personalization and project-based evaluation set it apart, and we monitor costs closely to keep the model sustainable.

A second risk is content quality. The AI can produce incorrect or outdated information, and sometimes adds unnecessary details. We plan to review and refine the content regularly to keep it accurate.

4.10.3 User Risks

Some users might find the platform difficult or lose motivation along the way. We address this with a clean, approachable design and an AI mentor that guides learners step by step.

4.10.4 Security Risks

Users log in and share personal information. Weak security can lead to data loss or exposure. We mitigate this through secure hosting, strong password policies, and protective measures across all user data.

Conclusion

This chapter covered the core specifications for GrowWise. It described what the system does, how users and admins interact with it, and how the components fit together. The functional and non-functional requirements are carefully defined so the platform stays fast, reliable, and easy to use. We discussed quality attributes such as performance, usability, and scalability, which all feed into the user experience. The chapter also covered assumptions, hardware and software needs, and likely development risks. On top of that, the use cases, UI designs, and database structure show how modules fit together and how the system operates end to end. Together, these specifications give us a clear plan for building, testing, and improving the platform.

Chapter 5 High-Level and Low-Level Design

This chapter tell high level and low level design of our project.

5.1 System Overview

GrowWise is intelligent AI learning application, which personalize the study to the developer. It inspect learner skill now, make special way to them and give smart assistance with Agentic RAG teacher inside. System care about teach that teaches how to think and do real project to make good problem solving skill.

5.1.1 Learning Management and Modules

GrowWise inserted content in small interactive lesson call learning node. Had explicit goal, example and actual task. AI make roadmap step by step user go. Check in on your preparedness before the next and auto monitor your performance.

5.1.2 Personalized Learning Paths

AI planner look your initial test and track that you choose. Make special roadmap, then, with you. When you practice change path and not how quickly you can learn, what you can do and what you require longer to learn. But you not study same, again or jump to too hard too quick.

5.1.3 Agentic RAG Mentor (Context-Aware Assistance)

You have intelligent AI teacher within app. It locate best resource to your lesson now and explain easy give when you need to. You inquire anything it respond question, provide explicit tip and display helpful thing that correspond to what you presently do and what you did in the past.

5.1.4 Assessment and AI Software Engineer Evaluation

You do quiz, small task and actual project challenge. Check your work like senior boss with one fake AI engineer. It examine your decision, good bad thing and code quality. Then provide feedback with idea how to improve.

5.1.5 Progress Insights and Feedback

Your personal dashboard displays your performance: How quick you are, what you good at and what still bad. This assist you practice on weak part, say what to do next and make your plan work you.

5.1.6 High-Level Technology View

Make new clean tool app: Next.js to look nice on the front, FastAPI to look fast on the back end, special AI part with LLM and RAG, and PostgreSQL to save everything, Supabase. Every service is deployed in Docker box and run on AWS server. Safe and sound, got good login and power control.

5.2 Design Considerations

In this section, main assume, limit and rule that make GrowWise design choice are discussed.

5.2.1 Assumptions and Dependencies

Related software and hardware

- AI service: The system relies on external LLM APIs such as OpenAI or Gemini, along with embedding models for the RAG pipeline. Normal response times and provider rate limits are assumed.
- Data layer: PostgreSQL on Supabase is used for structured data, with vector support for semantic search. Storage is also required for uploaded files and images.
- Hosting: The application runs on an AWS Linux x86-64 VPS using Docker containers. A GPU is not required at this stage, although local inference may be explored later.
- Client: Users need a modern browser such as Chrome, Firefox, or Safari with JavaScript enabled.

Operating systems

- Server: Linux is used in production, and the CI/CD pipeline also runs in a Linux environment.
- Developer computer: Development can be done on Windows 10 or 11, macOS, or Linux, with Docker Desktop available.
- Runtime base: Python 3.10 or later, Node.js 18 or later, and PostgreSQL 14 or later are required.

End-user characteristics

- Main people: The main users are junior developers with basic coding knowledge who can comfortably read English.
- Many user got slow or bad internet. The application should remain usable on slow or unstable internet connections and recover cleanly after interruptions.

- Easy use for all: The interface should support keyboard navigation, clear visual contrast, and screen reader accessibility.

Possible and probable changes in functionality

- Maybe change AI company or model. The system should support switching providers or models through an adapter-based integration layer with retry handling.
- New lesson and stuff add later. New lessons and learning features should be easy to add without major changes to the database or service structure.
- Test rule and feedback change time time. Assessment rules and feedback logic may change over time, so the evaluation pipeline should be versioned.

5.2.2 General Constraints

- Hardware or Software Environment: The application is expected to run on modest cloud infrastructure such as AWS or Render. Since CPU and memory are limited, the system should stay lightweight and efficient. The database and vector storage should support a typical volume of content and embeddings. No local GPU is assumed, so AI processing depends on external providers such as OpenAI or Gemini.
- End-User Environment: The platform should work in modern browsers on desktops, laptops, and mobile devices. It should remain usable on slow or unstable internet connections, with progress saved automatically. The interface should adapt to different screen sizes without losing usability.
- Standards Compliance: User privacy should be protected by storing only the data that is needed and securing it properly. The interface should support accessibility through readable typography, sufficient color contrast, and full keyboard navigation. Security practices should follow OWASP guidance for authentication and API key handling.
- Interface / Protocol Requirements: All communication should use HTTPS and JSON-based REST APIs. Sessions should be managed with secure JWT tokens. External API calls should include timeouts and retry logic where appropriate.
- Security Requirements: All data in transit should be protected with HTTPS. Passwords and tokens should be stored securely using strong hashing and protection mechanisms. Role-based access control should separate administrator and student permissions.
- Performance Requirements: Database queries should remain responsive under concurrent use. AI features such as plan generation and automated evaluation should scale as the number of learners

grows.

5.2.3 Goals and Guidelines

- KISS (Keep It Simple): The system should stay straightforward and avoid unnecessary complexity. A simpler design is faster to build, easier to maintain, and less likely to introduce bugs.
- Easy-to-Use, Friendly Interface: The application should feel simple and approachable. Learners should be able to use it with minimal guidance and stay motivated to continue.

5.2.4 Development Methods

GrowWise was developed in small, manageable modules using an agile process. This makes the system easier to adapt, extend, and maintain.

- Modular Architecture: The platform is divided into separate modules such as user authentication, learning plan generation, AI-based grading, and the RAG assistant. Each module can be developed, tested, and improved independently.
- Agile Methodology (Scrum-inspired): Development followed short iterations, regular demos, quick feedback cycles, and steady improvement.
- Backend Infrastructure: FastAPI provides the main REST backend and integrates with external services such as OpenAI, Gemini, and other APIs.
- Frontend Implementation: Next.js and React are used to build a responsive interface that works well across phones, tablets, and computers.
- Data Persistence: PostgreSQL stores structured application data, while vector support enables semantic search and RAG features.
- Version Management: GitHub (or GitLab) is used for collaboration, branching, and automated CI/CD workflows.
- Methodological Rationale: This combination supports flexibility, long-term maintenance, and future growth as the learning and AI requirements evolve.

5.3 System Architecture

This section describes the internal structure of the system modules and the external interactions with other systems. A diagram is included to show the overall architecture and how the parts connect.

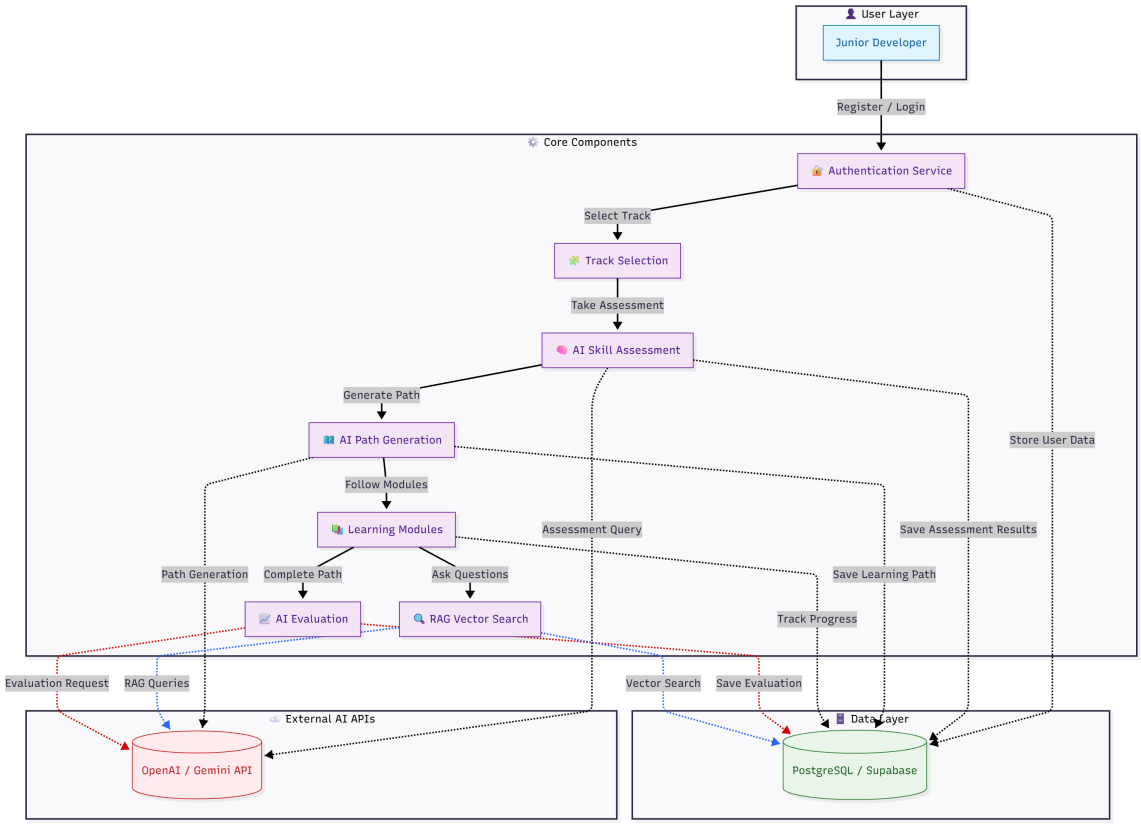

Figure 5.1: GrowWise architecture overview—showing interaction between frontend, backend, AI services, and database.

5.3.1 High-Level System Decomposition

The GrowWise platform is organized into four main parts, each responsible for a different area of the system:

- User Interface Layer: Manages user interactions and provides the main web interface
- Application Services Layer: Contains the core business logic and coordinates user activities
- AI Services Layer: Supports AI-driven features such as skill assessment, learning plan generation, and progress evaluation
- Data Persistence Layer: Handles the storage, retrieval, and organization of data

5.3.2 Major System Responsibilities

The software system is responsible for the following main functions:

1. User Management: Secure login, profile management, and session handling
2. Adaptive Learning: Real-time skill assessment and personalized learning plan generation
3. Content Delivery: Delivery of interactive lessons with built-in AI support

4. Intelligent Evaluation: AI-assisted review of user progress and project work
5. Progress Tracking: Recording learner activity and generating progress reports
6. Administrative Control: Managing lesson content, learning paths, and system settings

5.3.3 Component Roles and Responsibilities

5.3.3.1 User Interface Layer

- Authentication Module: Handles login, registration, and session management
- Track Selection Interface: Allows users to choose their learning focus area
- Assessment Interface: Presents skill assessments and collects user responses
- Learning Dashboard: Displays personalized learning plans and progress updates
- Module Interface: Provides an interactive learning space with built-in AI chat support
- Evaluation Interface: Supports project submission and displays feedback

5.3.3.2 Application Services Layer

- User Service: Manages user accounts and login rules
- Assessment Service: Handles skill tests and scoring
- Path Generation Service: Builds custom learning maps
- Module Service: Takes care of delivering lessons and tracking progress
- Evaluation Service: Organizes AI reviews for projects
- Admin Service: Gives tools for admins to manage content

5.3.3.3 AI Services Layer

- LLM Integration Service: Connects to external AI providers such as OpenAI or Gemini
- RAG System: Uses retrieval to add context for better AI help
- Vector Database: Keeps and searches embeddings of learning materials
- Assessment AI: Reviews answers and gives skill scores
- Path Planning AI: Generates flexible learning plans based on user information
- Evaluation AI: Provides project feedback in the style of a senior engineer

5.3.3.4 Data Persistence Layer

- User Database: Stores user information, authentication details, and preferences
- Content Database: Stores lessons, learning paths, and supporting resources
- Progress Database: Tracks user progress, completed work, and performance records
- Vector Store: Holds embeddings for quick content searches
- Session Storage: Manages short-term data and user sessions

5.3.4 Component Collaboration and Data Flow

The system components interact through clearly defined interfaces and data flows:

1. User Authentication Flow: Interface Layer → User Service → Database
2. Assessment Process: Interface Layer → Assessment Service → AI Services → Database
3. Path Generation: Assessment Service → Path Planning AI → Module Service
4. Learning Process: Module Service → RAG System → Vector Database → AI Services
5. Evaluation Process: Evaluation Service → Assessment AI → Database

5.3.5 Architectural Rationale

This decomposition was selected instead of alternative approaches for several reasons:

- Layered Architecture: Separates responsibilities clearly and makes the system easier to maintain
- Microservices Pattern: Allows AI services to scale and be deployed independently
- API-First Design: Makes integration with external AI providers and future extensions easier
- Stateless Services: Ensures scalability and fault tolerance
- Modular AI Integration: Allows easy switching between AI providers and models

Alternative monolithic approaches were rejected due to scalability limitations and tight coupling between AI and business logic components.

5.3.6 External System Integration

The system integrates with the following external services:

- OpenAI/Gemini APIs: For LLM-based AI functionality

- AWS VPS: For hosting and infrastructure services
- Supabase: For database and authentication services
- Vector Database Services: For embedding storage and retrieval

5.3.7 Subsystem Architecture

The AI Services Layer is main part of system. It handle all smart AI thing by connect to outside API. It got many small part that work together make good learning.

5.3.7.1 AI Assessment Module

This module check user skill use outside AI API. It make question from path user choose and send answer to external AI check. It get back score and weak spot then save in database for make plan. All depend on external AI for make question and grade.

5.3.7.2 AI Path Generation Module

This module make special plan for every user with help from outside AI. It take test result and send to external API with user path. AI think and send back lesson order with step and goal. Finally, save the plan in database so that system can use it.

5.3.7.3 Learning Module with RAG

This module supports the actual learning process with help from external AI services. When a user asks a question or requests assistance, the system sends the query together with relevant lesson context to the AI service. The response provides explanations and guidance based on the current lesson. The module also tracks user progress and completed activities.

5.3.7.4 AI Evaluation Module

This module evaluates user projects through external AI APIs. It sends submitted work to the AI service for analysis, then receives feedback, scoring, and suggestions for improvement. The goal is to provide responses that resemble the type of review a senior engineer might give.

5.3.7.5 External API Integration

All AI capabilities in the system rely on external providers such as OpenAI and Gemini. These APIs are used for assessments, learning path generation, RAG-based assistance, and project evaluation. Each

module is responsible for preparing requests correctly, handling responses, and managing errors. This approach adds intelligent functionality without requiring in-house model development.

5.3.7.6 Data Flow

User data moves through the AI layer, with external APIs handling the main AI-driven tasks. Assessment responses are sent for scoring, learning path requests are processed by the planning service, lesson questions are handled through the RAG workflow, and submitted projects are reviewed by AI. The results are then stored in the local database for future use and progress tracking.

5.4 Architectural Strategies

This section explains the main architectural choices made for the GrowWise platform.

5.4.1 Using External AI APIs

GrowWise uses external AI providers such as OpenAI and Gemini instead of building custom models from scratch. This allows the platform to access advanced AI capabilities without the cost and complexity of training, maintaining, and updating models internally. It also supports a pay-as-you-use model, which helps control operating costs. Building an in-house model would require substantial time, expertise, and infrastructure. By relying on external APIs, the project can focus more directly on the learning platform itself.

5.4.2 Using RAG for Smart Help

RAG, or Retrieval-Augmented Generation, is used to provide more relevant support to learners. It retrieves information from the system's knowledge base and combines it with AI-generated responses. This helps the platform deliver answers that are more accurate, contextual, and up to date. Compared with static help pages, RAG is easier to maintain and better suited to delivering support at the moment the learner needs it.

5.4.3 Choosing Modern Technology

Modern technologies such as Next.js for the frontend, FastAPI for the backend, and PostgreSQL for data management were selected for the platform. These tools integrate well with AI-based workflows and support reliable web application development. Older or less flexible technologies would have made integration and future development more difficult. The selected stack provides a practical balance of speed, maintainability, and scalability.

5.4.4 Using Microservices

The system follows a microservices-style structure in which major functions are separated into smaller services that communicate with each other. This makes it easier to update or improve one part of the system without affecting the rest. It also improves maintainability and reduces the impact of failures in individual services.

5.4.5 API-First Design

The platform is designed with an API-first approach. This makes it easier to integrate external AI providers, extend the system in the future, and support other clients such as mobile applications. As a result, the architecture remains flexible and better prepared for future expansion.

5.5 Domain Model/Class Diagram

The domain model and class diagram present the overall structure of GrowWise. They show the main system entities, their attributes and responsibilities, and the relationships between them. The model is organized into three connected views, each focusing on a different aspect of how the system operates. View 1: User Journey & Progress shows how a learner moves through the platform, from registration and path selection to skill assessment, plan generation, progress tracking, and reporting. This view captures the main user flow and the way the system responds to learner performance. View 2: AI Services & Knowledge Management focuses on the intelligent features that support personalized learning. It includes the RAG knowledge layer, vector storage, conversational AI support, learning path generation, and lesson assistance. View 3: Project Workflow & Evaluation describes how practical tasks are assigned, submitted, and reviewed. It shows how project work is evaluated by AI in a way that resembles engineering feedback and how that feedback supports improvement through applied learning. Taken together, these views provide a complete picture of GrowWise, including user interaction, data movement, and the role of AI in delivering personalized learning.

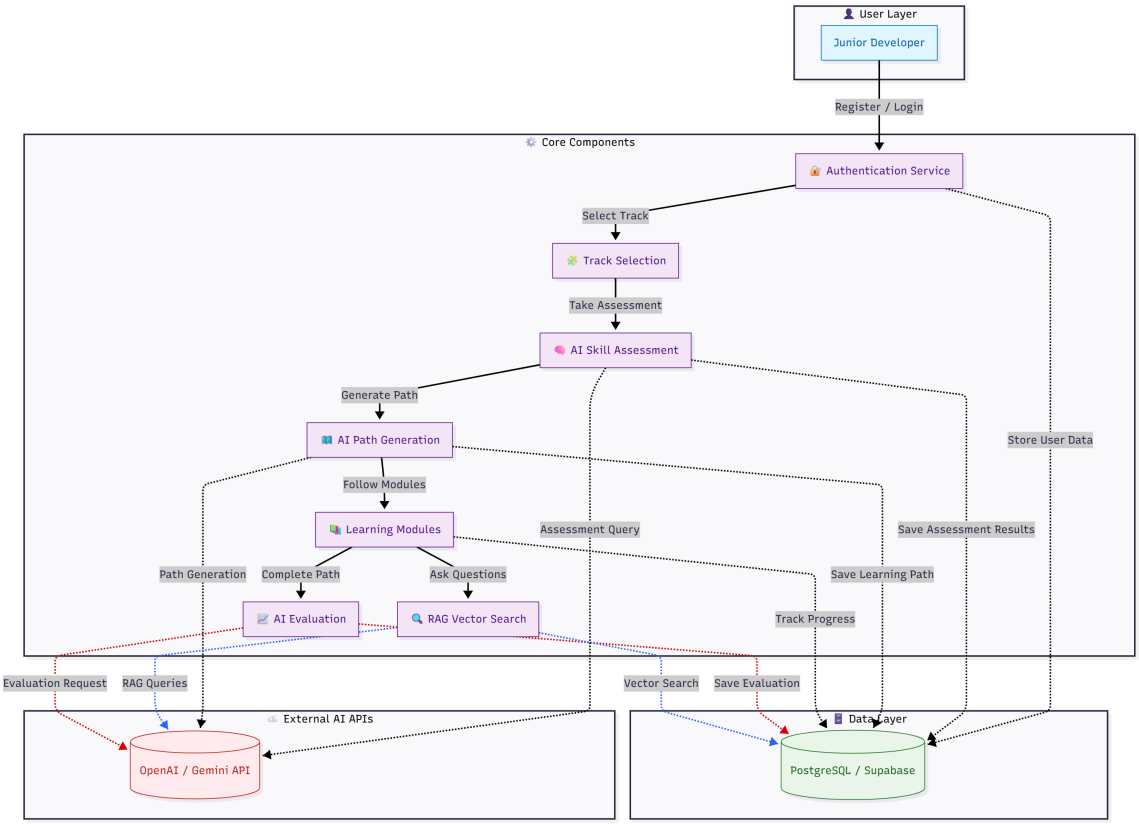

Figure 5.2: GrowWise system architecture: frontend, backend, AI, and database.

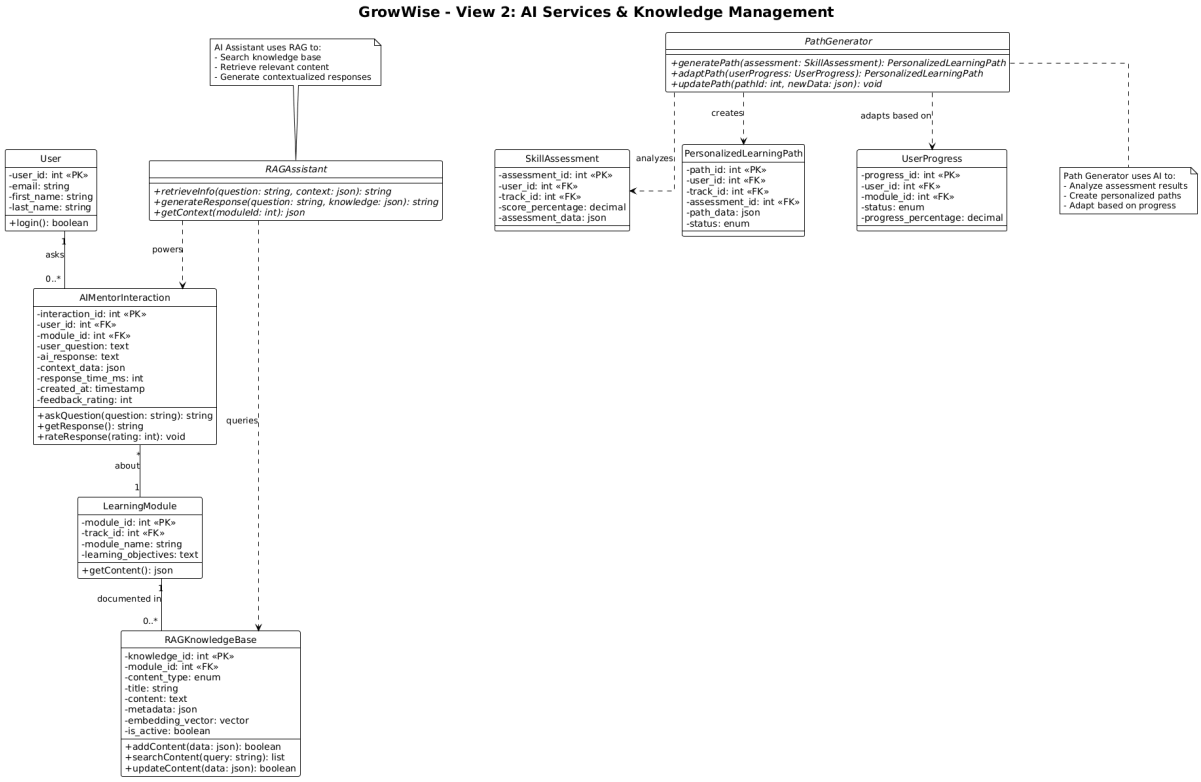

Figure 5.3: GrowWise – View 2: AI Services & Knowledge Management, highlighting components like the RAG knowledge base, contextual assistant, path generator, and AI mentor for adaptive, context-aware learning.

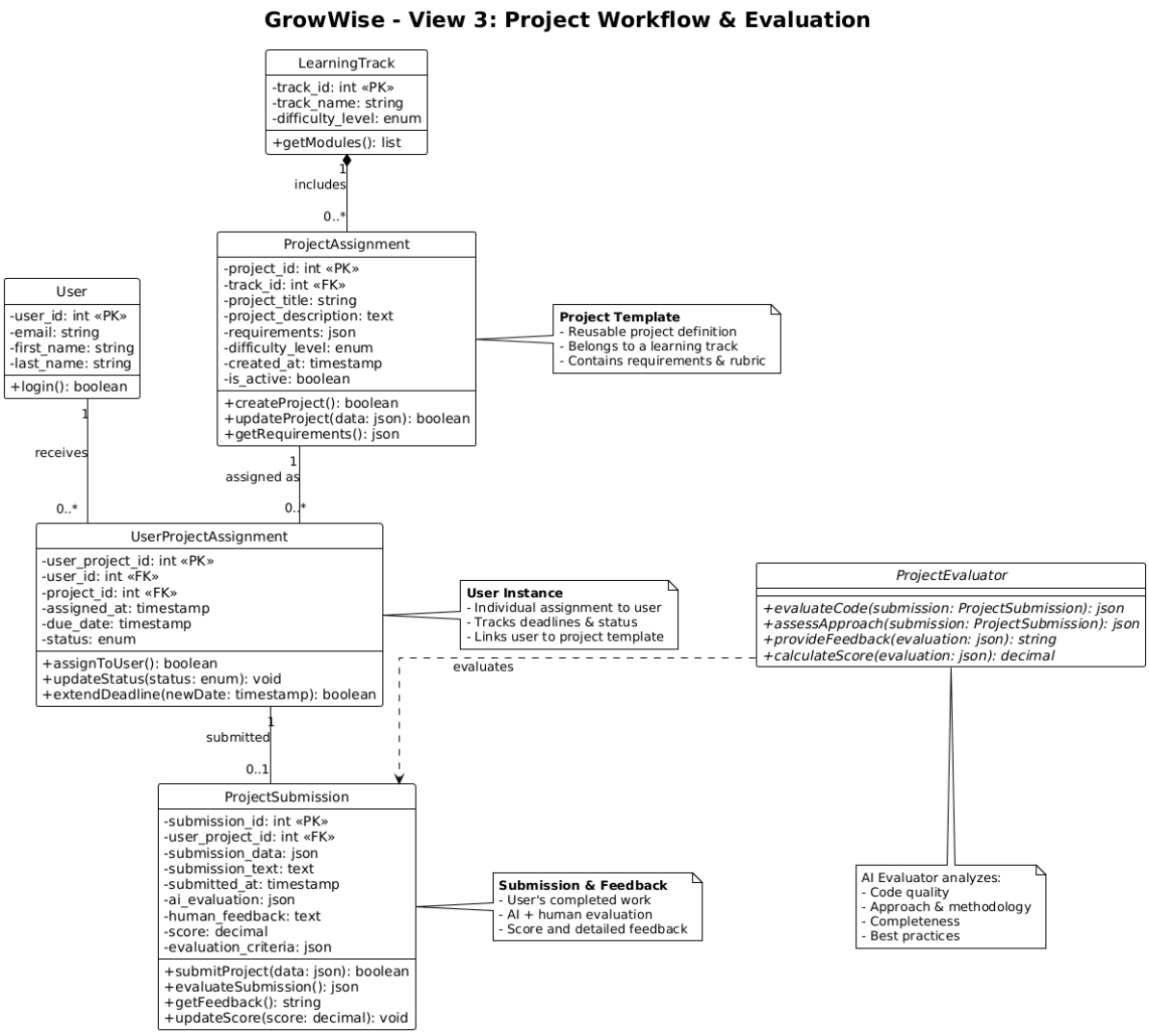

Figure 5.4: GrowWise – View 3: Project Workflow & Evaluation, showing how projects are assigned, submitted, and AI-evaluated with scoring and feedback.

5.6 Policies and Tactics

This section explains the exact design decisions and the implementation aspects that influence the work of the system.

5.6.1 Codebase Organization

Our code is well-organised in a simple folder structure. The primary project is divided into three major folders: frontend that contains the user-facing aspects, backend which contains server-related aspects, and shared which contains code which is shared between the two ends. We put similar files together just in the same folder. Similar, all of the code relating to logins in a single location, all lesson module code in a different location. In this manner, it is easy to locate things and update them. We use a regular name

of folders and files. Folders are small case with the use of underscores and files are similar to that. It assists in all getting the layout quickly.

5.6.2 Development Tools

We select specific instruments in constructing the application. VS Code is the primary choice of everyone because it is compatible with our languages. The version tracking is done by Git and the code is stored in GitHub. Docker deploys the app on local computers and in live environments to ensure that environments remain similar. The APIs are tested by Postman and the front end is debugged using Chrome DevTools. We also record everything so that new members can be on their feet fast. And automatic install and set up scripts.

5.6.3 Coding Guidelines

We possess simple coding regulations. Code must be well maintained and understandable. Instruments auto-make it appear homogenous. Classes and functions are assigned meaningful names. We add notes for tricky parts. In Python, type hints are used to explain things. Code before merge is auto checked to be compliant to rules.

5.6.4 Testing Strategies

There are various ways of testing code. Unit tests are analyzed of individual functions, the integration tests of how components are linked together and end-to-end tests of the complete user paths. Tests are made of new features and fixes. They have to die before they can unite. We monitor the extent of code tests and are ambitious. Automated applications will perform tests on all the changes and this will ensure that things are always working and that any problems are identified early.

5.6.5 Documentation Practices

We document everything well. The APIs receive clear definitions on the uses and how they work. Complex bits have code notes. We maintain platform user guides and dev guides of the code. In addition, deployment instructions and troubleshooting manuals. We update documentations with the changes and go through them regularly to ensure that they are accurate and useful.

5.7 Conclusion

This chapter presented the complete design specification for the GrowWise AI-based adaptive learning platform, covering the main architectural decisions and implementation plan. It began with an analysis

of the system requirements and constraints, which guided the development of a scalable and extensible structure for developer-focused learning. The proposed architecture combines Next.js for the frontend, FastAPI for backend services, PostgreSQL for data management, and external AI APIs for learning path generation, guidance, and evaluation. The design supports adaptive learning through dynamic path generation, real-time AI assistance using RAG, and detailed assessment of technical and problem-solving skills. These decisions were made to balance scalability, maintainability, usability, security, performance, and accessibility. The modular structure also allows each part of the system to be developed, tested, and improved independently. Supporting documentation, including the class diagram, system interactions, and architectural model, provides a strong foundation for implementation. Overall, the design meets the main objective of creating a flexible and personalized learning platform that goes beyond fixed courses and adapts to the needs and learning styles of individual developers.

Chapter 6 Implementation and Testing

The chapter addresses the process of creating and testing the GrowWise prototype. It contains the technology selections, determining the particulars of the keybuilds of the parts, the connections with AI, data archives, test arrangement, and test results, launch, and present constraints.

6.1 Implementation

The following are the construction specifications of the components and procedures of the plan.

6.1.1 Learning Track and Module System

The Learning Track and Module System establishes the framework of the simple system of GrowWise, with lessons arranged into definite sequences. Big tech areas are referred to as tracks and modules are the units within the tracks. To begin with, we have created a flexible database structure consisting of multitrack and ordered modules. The back end is a FastAPI SQLAlchemy based implementation of LearningTrack and LearningModule items. Songs are listed on a table named, described, difficulty, and time estimate. Tracks are interconnected through a key to a module and order exists. The next.js and React front end display tracks in cards which contain information such as description, difficulty, and time. The selection of a track stores a UserTrackSelection link. The module view presents information sequentially with text, videos, coding assignments, and quizzes. It verifies requirements to maintain the appropriate sequence. Modules are loaded depending on the user spot in the path. UserProgress in progress tracks including status, time and percent. Admins have the ability to revise content without discontinuing progress, quality versions.

6.1.2 AI-Powered Skill Assessment Module

The AI-Powered Skill Assessment Module tests to determine user knowledge and problem-solving to identify the optimal start. It combines smart grading with AI generated questions. The track is made by using outside AI APIs such as OpenAI or Gemini to make questions. The back end receives track information, constructs a prompt that the AI will use to generate questions on knowledge and methods. It establishes types, problems and coverage. There are questions that have options of facts, reasoning questions, and practice questions in terms of their coding. Every save has its track, difficulty and criteria. We retain a bank to be reused, and make others to personal checks. In grading, questions and answers are submitted to the AI using back end with the prompt to verify the correctness, reasoning and understanding. It identifies areas of weaknesses, strengths, and provides feedback on techniques.

Scores, JSON, and time of results are stored in SkillAssessment. AI information informs the decisions of making a path, closing holes, and creating strengths.

6.1.3 AI Learning Path Generation Module

The AI Learning Path Generation Module creates personalized road maps based on the assessments and creates the order of the best modules. It encompasses information analysis and artificial intelligence design. It starts after assessment. Results such as scores, gaps, strengths and track are obtained through back end. This passes on to AI with module data, requirements, challenges, and objectives. AI is the planner based on immediate information on the user, modules, connections and regulations. It tracks a route using systematized modules, areas of focus, times and milestones. Path It is a JSON containing module IDs, positions, prerequisites, times and objectives. It consists of adaptive features such as reviews, additional practice and advanced options. It Stores the PersonalizedLearningPath as a flexible JSON file. UserProgress System watches move forward in the path, performance check, and makes the path refined with AI when necessary. It monitors progress, speed and problem areas. Massive alterations cause path updates to align with actual learning. Path is displayed as interactive map on front end where the nodes represent modules and display the status. It has used percent, time remaining and milestones. Position, links, and module details are displayed to the users.

6.1.4 RAG Implementation for Integrated Learning Sources Context

The Retrieval-Augmented Generation (RAG) system is developed on the lines of the smart system that retrieves learning materials to form a knowledge base rich in context. This was constructed in a stepwise fashion that manages materials, constructs semantic embeddings, and mining out pertinent content intelligently. It begins with content importation, during which we collect information such as documentation, tutorials, code snippets, explanations and documentation and prepare it. The back end contains uploading file, picking type and linking to modules administrative tool. It works with PDF, markdown, text and JSON. The processing is a breaking down of docs into meaningful pieces. We cut smartly, but we still keep structure structured such as paragraphs, headers, code, sentences, etc. as opposed to cutting by size. Every work is converted to embeddings with the models of OpenAI such as text-embedding-ada-002 to obtain vectors that encode the meaning. This facilitates the process of locating the related concepts, not only the identical words. To enable us to find the vectors quickly in the database, we use PostgreSQL and pgvector. The pieces are stored in the RAGKnowledgeBase with their vectors, source information, type, link to a module, and some time. Indexes also enable rapid searches with large quantities of data. Retrieval combines semantic search and filters. In the case of user query, we embed the query, and find similar vectors by cosine. We are able to narrow down to the module or type. We prioritize the best

pieces, take the most few such as 3-5 and submit them along with the question to the LLM. The prompt will direct the AI to apply that information to give precise and cordial responses such as a mentor. We made our adjustments towards enhanced speed and precision. Caching is the records of frequent questions and descriptions so as to reduce the number of calls and accelerate the process and time-outs are used to refresh it. We extend queries with module background and user history also. All the mentor chats are logged in AIMentorInteraction, including the question, chunks, response, time and feedback. This assists in identifying patterns, missing knowledge and enhancing retrieval. Each module of the front end possesses a smooth chat that displays history, indicates natural questions and clear responses. It proposes questions, displays content that is referred to, and allows users to rate responses.

6.1.5 AI Evaluation Module

After successfully going through each module and its content, we implemented a Gemini-based chatbot AI evaluator. It has complete context about the track, user journey, and the educational content the user has consumed and studied.After that, the AI works like a reverse ChatGPT, asking the user questions through a chatbot. Once the session ends, it provides a complete progress report to the end user. In this report, it clearly shows the user's level when they started and what their current level is now.

6.2 Test case Design and description

This section presents the test cases designed and executed for each major functional module of the GrowWise platform. Common shared attributes across all test cases are: a modern web browser with JavaScript enabled, the FastAPI backend running on the configured port, a PostgreSQL database connected, and internet access for AI API calls. Each test case follows the standardised format and was verified against the fully implemented system.

Table 6.1: Test Case TC-AUTH-01: User Login Authentication

Authentication & Account Security Module			
GrowWise v1.0 – server/app/routers/auth.py			
Test Case ID:	TC-AUTH-01	**QA Test Engineer:**	M. Umair Imran
Test case Version:	1.0	**Reviewed By:**	Maryam Nasim
Test Date:	10/03/2026	**Use Case Ref:**	UC-01 User Login
Revision History:	No prior revision history.		
Objective:	Verify a registered user can log in with valid credentials and receive a JWT access token.		
Product/Ver/ Module:	GrowWise v1.0 / Authentication Module		
Environment:	Web browser, FastAPI backend, PostgreSQL, Python 3.10+, Node.js 18+		
Assumptions:	User account exists in the database. Backend server is running.		
Pre-Requisite:	A registered user account with a valid email and password must exist.		
Step No.	**Execution Description**	**Procedure Result**	
1	User navigates to the login page.	Login form displays email and password input fields.	
2	User enters valid credentials and clicks Submit.	System validates credentials and issues JWT access and refresh tokens.	
3	System redirects user to the dashboard.	Dashboard loads with the authenticated user profile data.	
3-A	User enters an incorrect password.	System shows “Incorrect email or password” error message.	
Comments: Both valid and invalid credential flows tested and verified.			
Passed Failed Not Executed			

Table 6.2: Test Case TC-TRACK-01: Learning Track Selection

Track Selection Module			
GrowWise v1.0 – server/app/routers/tracks.py			
Test Case ID:	TC-TRACK-01	**QA Test Engineer:**	M. Shahzad Waris
Test case Version:	1.0	**Reviewed By:**	Maryam Nasim
Test Date:	10/03/2026	**Use Case Ref:**	UC-02 Track Selection
Revision History:	No prior revision history.		
Objective:	Verify user can browse available learning tracks and select one to begin personalized learning.		
Product/Ver/ Module:	GrowWise v1.0 / Track Selection Module		
Environment:	Web browser, FastAPI backend, PostgreSQL database, Python 3.10+		
Assumptions:	User is authenticated. At least one learning track exists in the database.		
Pre-Requisite:	User must be logged in. Tracks must be available in the system.		
Step No.	**Execution Description**	**Procedure Result**	
1	User navigates to the track selection page.	Available learning tracks displayed as selectable cards.	
2	User selects a learning track (e.g., Web Dev).	Track saved to UserTrackSelection table. Confirmation displayed.	
3	System redirects to the skill assessment.	User routed to assessment page for the chosen track.	
3-A	User proceeds without selecting any track.	System prompts: “Please select a track to continue”.	
Comments: Correct track saved to database and assessment session correctly initiated.			
Passed Failed Not Executed			

Table 6.3: Test Case TC-ASSESS-01: AI Skill Assessment

AI Skill Assessment Module			
GrowWise v1.0 – server/app/routers/assessment.py			
Test Case ID:	TC-ASSESS-01	**QA Test Engineer:**	Ameer Tufail
Test case Version:	1.0	**Reviewed By:**	Maryam Nasim
Test Date:	11/03/2026	**Use Case Ref:**	UC-03 Skill Assessment
Revision History:	No prior revision history.		
Objective:	Verify AI generates track-specific questions and evaluates answers to produce a skill score and learning path.		
Product/Ver/ Module:	GrowWise v1.0 / AI Skill Assessment Module		
Environment:	Web browser, FastAPI, Gemini/OpenAI API, PostgreSQL database		
Assumptions:	Learning track is selected. AI API is accessible. Mock AI flag is disabled.		
Pre-Requisite:	User must have a selected learning track before starting the assessment.		
Step No.	**Execution Description**	**Procedure Result**	
1	System creates an assessment session.	10 AI-generated questions returned and displayed to the user.	
2	User answers all questions and submits.	Answers saved to the assessment responses table in the database.	
3	User clicks Complete Assessment.	AI batch-evaluates; skill score, report, and learning path generated.	
3-A	User attempts to complete with unanswered questions.	System prompts user to answer all remaining questions first.	
Comments: AI response quality and mock fallback mode both tested and verified.			
Passed Failed Not Executed			

Table 6.4: Test Case TC-LPATH-01: AI Learning Path Generation

AI Learning Path & Stage Management Module			
GrowWise v1.0 – server/app/routers/learning.py			
Test Case ID:	TC-LPATH-01	**QA Test Engineer:**	M. Umair Imran
Test case Version:	1.0	**Reviewed By:**	Maryam Nasim
Test Date:	11/03/2026	**Use Case Ref:**	UC-04 Path Generation
Revision History:	No prior revision history.		
Objective:	Verify AI generates a personalized ordered stage-based learning path after assessment completion.		
Product/Ver/ Module:	GrowWise v1.0 / AI Learning Path Module		
Environment:	FastAPI backend, Gemini/OpenAI API, PostgreSQL, Next.js frontend		
Assumptions:	Assessment is completed and result is stored in the database. AI API is accessible.		
Pre-Requisite:	Skill assessment must be completed; result stored in SkillAssessment table.		
Step No.	**Execution Description**	**Procedure Result**	
1	Assessment completion triggers path generation.	AI receives user score, skill gaps, strengths, and track data.	
2	AI generates an ordered list of learning stages.	Path saved as JSON in PersonalizedLearningPath table.	
3	User views roadmap on the dashboard.	Stages shown in correct order with status and progress indicators.	
3-A	Path generation fails due to AI API error.	System shows “Try assessment again or contact support”.	
Comments: Path verified to target weak areas identified during assessment.			
Passed Failed Not Executed			

Table 6.5: Test Case TC-CONTENT-01: Stage Content Generation and Progress Tracking

Learning Content Generation & Progress Module			
GrowWise v1.0 – server/app/routers/content.py			
Test Case ID:	TC-CONTENT-01	**QA Test Engineer:**	M. Shahzad Waris
Test case Version:	1.0	**Reviewed By:**	Maryam Nasim
Test Date:	12/03/2026	**Use Case Ref:**	UC-11 Module Access
Revision History:	No prior revision history.		
Objective:	Verify AI generates structured stage content and user progress is tracked correctly upon completion.		
Product/Ver/ Module:	GrowWise v1.0 / Content Generation & Progress Module		
Environment:	FastAPI, Gemini API with Google Search grounding, PostgreSQL		
Assumptions:	Learning path exists and the target stage is unlocked for the user.		
Pre-Requisite:	Personalized learning path must be generated; user must be on an active stage.		
Step No.	**Execution Description**	**Procedure Result**	
1	User opens a learning stage.	Content generation request sent; AI fetches and structures the content.	
2	AI generates content using search-grounded results.	Structured content saved to stage content table and displayed.	
3	User marks the stage as complete.	User progress updated to 100%. Next stage unlocked automatically.	
3-A	User accesses a stage with an invalid ID.	System returns a 404 error with a descriptive error message.	
Comments: Content quality verified. Progress persists correctly on page refresh.			
Passed Failed Not Executed			

Table 6.6: Test Case TC-RAG-01: RAG Mentor Chat Interaction

RAG Mentor Chat Module			
GrowWise v1.0 – server/app/routers/chat.py			
Test Case ID:	TC-RAG-01	**QA Test Engineer:**	Ameer Tufail
Test case Version:	1.0	**Reviewed By:**	Maryam Nasim
Test Date:	12/03/2026	**Use Case Ref:**	UC-05 RAG Assistance
Revision History:	No prior revision history.		
Objective:	Verify AI mentor provides context-aware answers by retrieving knowledge base content via the RAG pipeline.		
Product/Ver/ Module:	GrowWise v1.0 / RAG Mentor Chat Module		
Environment:	FastAPI, external RAG API, Gemini/OpenAI, PostgreSQL with pgvector		
Assumptions:	User is in an active learning stage. External RAG API is accessible.		
Pre-Requisite:	Active learning stage must exist. Knowledge base must have entries for the track.		
Step No.	**Execution Description**	**Procedure Result**	
1	User opens mentor chat for current stage.	Chat session created and scoped to the current stage context.	
2	User sends a question about the topic.	Query sent to RAG system; semantic retrieval performed on knowledge base.	
3	RAG returns top matching knowledge chunks.	AI generates and displays a context-aware answer to the user.	
3-A	External RAG API is unavailable.	System falls back to a base AI response without RAG context.	
Comments: Retrieval verified to return correct module-relevant content chunks.			
Passed Failed Not Executed			

Table 6.7: Test Case TC-EVAL-01: AI Evaluation Validator Interview

AI Evaluation Module – Validator Interview			
GrowWise v1.0 – server/app/routers/evaluation.py			
Test Case ID:	TC-EVAL-01	**QA Test Engineer:**	M. Umair Imran
Test case Version:	1.0	**Reviewed By:**	Maryam Nasim
Test Date:	13/03/2026	**Use Case Ref:**	UC-06 Project Assignment
Revision History:	No prior revision history.		
Objective:	Verify AI conducts a structured interview and produces a scored report on reasoning, problem-solving, and job readiness.		
Product/Ver/ Module:	GrowWise v1.0 / AI Evaluation Module		
Environment:	FastAPI, Gemini API, PostgreSQL, React frontend (Validator page)		
Assumptions:	Required learning modules are completed. AI API is available and responding.		
Pre-Requisite:	Required modules must be completed. No active evaluation session must already exist.		
Step No.	**Execution Description**	**Procedure Result**	
1	User starts an evaluation session.	AI sends introductory question with full track and path context.	
2	User completes minimum required dialogue turns.	Each exchange logged; AI asks contextual follow-up questions.	
3	User clicks Complete Evaluation.	AI scores reasoning and job readiness; evaluation report generated.	
3-A	User tries to complete before minimum dialogues.	Complete button stays disabled until minimum threshold is met.	
Comments: Minimum dialogue enforcement and evaluation scoring quality verified.			
Passed Failed Not Executed			

Table 6.8: Test Case TC-PROGRESS-01: Progress Insights and Improvement Analysis

Progress Insights, Reports & Improvement Analysis Module			
GrowWise v1.0 – server/app/routers/progress.py			
Test Case ID:	TC-PROGRESS-01	**QA Test Engineer:**	Ameer Tufail
Test case Version:	1.0	**Reviewed By:**	Maryam Nasim
Test Date:	13/03/2026	**Use Case Ref:**	UC-10 User Progress
Revision History:	No prior revision history.		
Objective:	Verify dashboard shows accurate progress metrics and improvement analysis displays correct before-vs-after skill comparison.		
Product/Ver/ Module:	GrowWise v1.0 / Progress & Insights Module		
Environment:	FastAPI, PostgreSQL, React with Recharts, Gemini API for analysis		
Assumptions:	User has completed at least one assessment and one evaluation session.		
Pre-Requisite:	Path completion report and evaluation result must exist in the database.		
Step No.	**Execution Description**	**Procedure Result**	
1	User opens the dashboard.	Progress charts, completion percentage, and key metrics displayed.	
2	User requests improvement analysis.	AI generates before-vs-after skill comparison with structured data.	
3	Report displayed with cards, charts, and tips.	Data matches stored assessment and evaluation scores in the database.	
3-A	No progress data exists for the user.	System shows an empty state with a helpful guidance message.	
Comments: Chart values verified against raw database scores. Empty state handled correctly.			
Passed Failed Not Executed			

6.3 Test Metrics

This section summarizes the quantitative metrics for all test cases executed across the GrowWise platform. Three views are provided: an overall summary, a module-wise execution breakdown, and a requirements traceability matrix.

6.3.1 Test Metric No.1: Overall Test Summary

Table 6.9: Overall Test Metrics Summary

Metric	Value	Description
Total Test Cases Designed	8	One test case designed per major system module
Total Test Cases Executed	8	All designed test cases were executed
Number of Test Cases Passed	8	All test cases produced the expected results
Number of Test Cases Failed	0	No test cases failed after final system testing
Test Cases Not Executed	0	All designed cases were fully executed
Pass Rate	100%	(Passed ÷ Executed) × 100 = (8/8) × 100
Test Case Defect Density	0%	(Failed ÷ Executed) × 100 = (0/8) × 100
Test Case Effectiveness	100%	All defects found during testing were detected by the test cases
Requirement Coverage	100%	All 8 functional requirement areas covered by at least one test case
Traceability Matrix	Covered	Each requirement has a corresponding test case (see Metric No.3)

6.3.2 Test Metric No.2: Module-Wise Test Execution Results

Table 6.10: Module-Wise Test Execution Results

Module Name	Designed	Executed	Passed	Failed	Pass %
Authentication & Account Security	1	1	1	0	100%
Track Selection	1	1	1	0	100%
AI Skill Assessment	1	1	1	0	100%
AI Learning Path Generation	1	1	1	0	100%
Content Generation & Progress	1	1	1	0	100%
RAG Mentor Chat	1	1	1	0	100%
AI Evaluation (Validator)	1	1	1	0	100%
Progress Insights & Analysis	1	1	1	0	100%
Total	**8**	**8**	**8**	**0**	**100%**

6.3.3 Test Metric No.3: Requirements Traceability Matrix

Table 6.11: Requirements Traceability Matrix

Requirement ID	Functional Requirement	Test Case	Status
FR-AUTH-01	User login and JWT token issuance	TC-AUTH-01	Covered
FR-TRACK-01	Learning track selection and storage	TC-TRACK-01	Covered
FR-ASSESS-01	AI question generation and batch scoring	TC-ASSESS-01	Covered
FR-LPATH-01	Personalized learning path generation	TC-LPATH-01	Covered
FR-CONTENT-01	AI stage content generation and progress tracking	TC-CONTENT-01	Covered
FR-RAG-01	RAG-based context-aware mentor chat responses	TC-RAG-01	Covered
FR-EVAL-01	AI evaluation interview and scored report	TC-EVAL-01	Covered
FR-PROGRESS-01	Progress dashboard and improvement analysis	TC-PROGRESS-01	Covered

6.3.4 Conclusion

This chapter covered build decisions and tested how it works on various levels. The results demonstrate that a flexible learning flow supported by AI is achievable and preconditions further construction and development.

Chapter 7 User Manual

7.1 Introduction

This chapter provides a complete step-by-step guide for using the GrowWise AI-powered adaptive learning platform. It covers every feature of the application from account creation to completing a full personalized learning journey. Any user, whether a beginner or an experienced developer, can follow this manual to navigate and use the platform without prior knowledge of its internals.

7.2 System Requirements

Before accessing GrowWise, ensure the following are in place:

- A modern web browser: Google Chrome, Mozilla Firefox, Microsoft Edge, or Safari (latest stable version recommended).
- JavaScript must be enabled in the browser.
- A stable internet connection is required at all times, as the platform relies on live AI API calls.
- No additional software installation is needed. The platform runs entirely in the browser.

7.3 Accessing the Platform

Open your web browser and navigate to the GrowWise application URL. You will be presented with the Home page. From here you can:

- you can click on **Take Free Assessment** or **Get Started** button for start using the system. if you dont have account yet then it will takes you to the Sign Up page.
- if you already login before then just click **Go to Dashboard** and it will go to your dashboard directly.
- there is also a **Blog** option in the footer area, you can click it for reading the articles that is available on this platform.

7.4 Sign Up

To create a new GrowWise account, you have to follow these steps:

1. First go to the Home page and click **Take Free Assessment** button, or you can also go directly to

`/signup` page.

2. After that the Sign Up page will open and you will see a heading that says *Join Grow Wise*.
3. Now you have to fill all the fields that is given there:
   - **Full Name**: write your full name here.
   - **Email**: put a valid email address because it will use for login and also for reset password.
   - **Password**: it must be atleast 8 characters long.
4. you also have to tick the **Terms and Conditions** checkbox otherwise the **Create Account** button will not become active, it stay disable until you check that box.
5. then click **Create Account** button. it will showing a loading for some time while it processing your request.
6. if everything is ok then it will automatic login you and take you back to Home page. also a small toast message will come like *"Account ready, {your name}"*.
7. but if the automatic login not working after register, then a success screen will appear and there will be a **Proceed to Login** button, you can click it for login manually.

7.5 Login

To log in to an existing GrowWise account:

1. Navigate to `/login` or click **Log in** from the Sign Up page.
2. Enter your registered **Email** and **Password**.
3. Click **Log In**. The button shows a loading indicator while verifying credentials.
4. On success, you are redirected to the Home page with a toast notification: *"Welcome back, {your name}"*.

As shown in Figure 4.1, the login screen displays the email and password fields clearly.

7.5.1 Forgot Password

If you have forgotten your password:

1. On the Login page, click **Forgot your password? Reset it**.
2. Enter your registered **Email** address.

3. Click **Send reset instructions**.
4. A message will appear: *"If an account exists for this email, a reset link has been sent."*
5. Check your email inbox for the reset token.

7.5.2 Reset Password

Once you have received a reset token by email:

1. Navigate to `/reset-password` or click **Already have a reset token?** on the Forgot Password page.
2. Paste your **Reset token** into the token field.
3. Enter your **New password** (minimum 8 characters).
4. Re-enter the same password in **Confirm new password**.
5. Click **Reset password**.
6. On success: *"Password reset complete. You can now sign in with the new password."* Return to the Login page and sign in with your new password.

7.6 Selecting a Learning Track

After logging in, the first step in your learning journey is choosing a track.

1. Click **Take Free Assessment** on the Home page, or click **Choose Track** from any page in the top bar.
2. The **Select Your Track** page loads, showing all available learning tracks as cards (e.g., Web Development, AI/ML, Cybersecurity).
3. Each card displays the track name and a short description.
4. Click **Start Assessment** on the track you want to begin.
5. A full-screen overlay will appear: *"Generating Your Assessment... "* The system is creating your personalised skill test.
6. You will be automatically redirected to the Assessment page once the questions are ready.

Notes:

- You can only start one assessment at a time; other track buttons are disabled while one is loading.

- If tracks fail to load, a **Retry** button is shown. Click it to reload.

As shown in Figure 4.2, the track selection screen presents all available tracks clearly.

7.7 Skill Assessment

The skill assessment measures your current knowledge and problem-solving ability for the chosen track. It is used to generate a personalised learning path.

1. The assessment page loads with up to **10 questions** generated by AI for your selected track.
2. A **50-minute countdown** timer appears in the top-right corner.
3. The current question number and a dot-progress bar are shown at the top.
4. Each question displays:
   - A **difficulty badge** and **question type badge**.
   - The question text (rendered with markdown formatting).
5. Answering questions:
   - **Multiple choice (MCQ):** Click one of the option buttons (A, B, C, D). Clicking an option submits it automatically and moves to the next question.
   - **Written answer:** Type your answer in the text box and click **Submit Answer**. The button is disabled until you type something.
6. After the last question is answered, the assessment is submitted automatically. A full-screen overlay shows *"Finalizing assessment…"*.
7. The timer reaching zero also triggers automatic submission.
8. On completion, you are redirected to the **Dashboard** and a toast notification confirms the assessment is complete.

Quitting early:

- Click **Quit** (top-right) to open a modal.
- Choose **Resume** to continue or **Exit** to leave. Unanswered questions will not be submitted if you exit early.

As shown in Figure 4.3, the assessment interface presents questions with a timer and progress tracker.

7.8 Dashboard

The Dashboard is your central hub for tracking progress, reviewing assessments, and navigating to your learning path. It is accessible from the left sidebar under **Overview**.

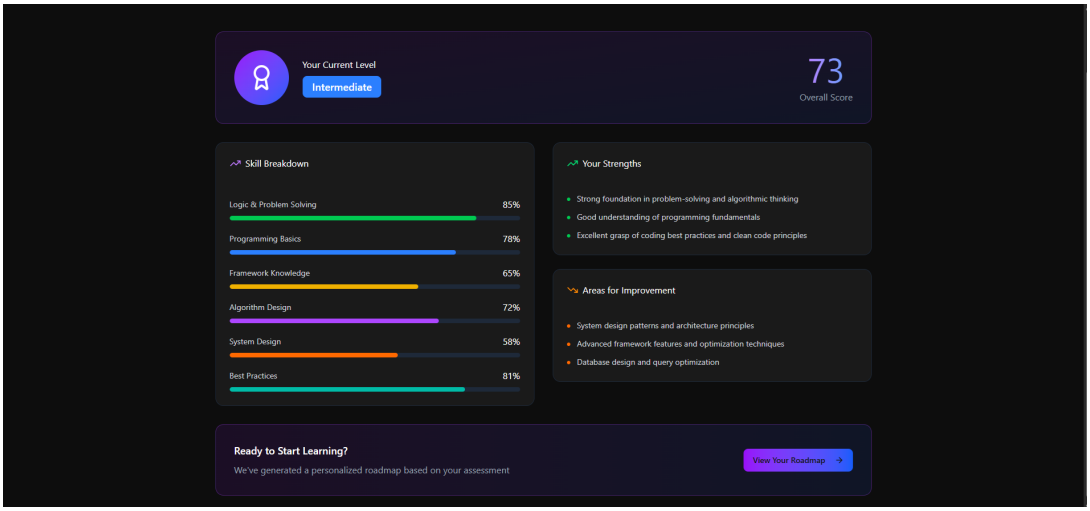

Figure 7.1: Dashboard showing the assessment report and progress overview.

7.8.1 Stat Cards

At the top of the Dashboard, four summary cards are displayed:

- **Assessments**: total number of assessments taken.
- **Latest Score**: your most recent assessment score percentage.
- **Learning Completion**: percentage of your current learning path completed.
- **Learning Time**: total hours spent learning.

7.8.2 Activity Timeline

Below the stat cards, an activity chart shows your learning activity over the selected time window (Last 7, 30, or 90 days). Use the **dropdown** in the top bar to switch between time windows. Click the **Refresh** icon to reload all dashboard data.

7.8.3 Assessment History and AI Report

1. The left column lists all your past assessment sessions by date and track.
2. Click any row to load the **AI Assessment Report** for that session on the right side.
3. The report shows: executive summary, overall assessment, dimension scores, and learning priori-

ties.

4. At the bottom of the report, click **Continue Learning** to open your personalised learning path.

7.8.4 Comparing Two Assessments

1. In the assessment history list, tick the **checkbox** on two different sessions.
2. The **Compare Selected** button (above the list) becomes active.
3. Click **Compare Selected** to open a modal showing a side-by-side comparison of the two attempts.
4. Close the modal by clicking the **X** button.

7.8.5 Path Completion Banner

When your full learning path is complete, a banner appears at the top of the Dashboard with:

- A snippet of your completion report.
- **Progress Analysis**: navigates to your Improvement Analysis page.
- **View Full Report**: opens the learning summary modal.

7.9 Learning Path

The Learning Path page (accessible via **Learning Path** in the left sidebar) shows your AI-generated personalised curriculum organised into stages. See Figure 4.5 for the roadmap view.

7.9.1 Navigating Stages

1. The left column lists all stages numbered in order (e.g., *1. Introduction to APIs*, *2. Backend Fundamentals*).
2. Click any stage name to load it in the main panel.
3. The selected stage shows its title, focus area, and a progress grid (completion %, items done, time spent).

7.9.2 Generating Stage Content

When a stage has no content yet:

1. A **Generate Content** button or **Generate Stage Content** prompt will appear in the stage panel.

2. Click it. The system will use AI with live search grounding to create 8 structured content items for that stage.
3. Wait for the content to load (this may take a few seconds as AI generates it).
4. Each content item shows: title, description, type (video / text / interactive), difficulty, estimated duration, and a reading/viewing body.
5. Some items include an **Open resource** link. Click it to open the external learning resource in a new tab.

As shown in Figure 4.6, the learning module displays AI-generated content alongside the mentor chat.

7.9.3 Tracking Your Progress

For each content item in a stage:

1. A **progress strip** at the bottom shows your current status: *Not started*, *X% complete*, or *Completed*.
2. If not started, click **Start Tracking** to begin timing your session.
3. If in progress, use:
   - **Update (+25%, +10m)**: logs 25% more completion and 10 additional minutes.
   - **Mark Complete**: sets the item to 100% complete.
4. Once all content items in all stages are marked complete, a **completion banner** appears at the top of the page.

7.9.4 Using the AI Mentor Chat

The right column on the Learning Path page contains the **AI Mentor Chat**. This is your context-aware AI assistant for the current stage.

1. Type your question about the current topic in the text field at the bottom of the chat.
2. Press **Enter** or click **Send** to submit your question.
3. The AI Mentor will search the knowledge base (RAG system) for relevant content and respond with a context-aware answer.
4. Previous messages are shown in the chat history above.
5. If the mentor is responding, a *"Mentor is responding... "* indicator appears.

Tips:

- Ask specific questions about the topics in the current stage for the best answers.
- If the RAG system is temporarily unavailable, the mentor will still respond using its base AI knowledge.

7.9.5 Completing the Learning Path

When all stages and content items are done:

1. A completion banner appears at the top of the Learning Path page.
2. Click **View Learning Report** to open a modal summarising your full learning journey.
3. Click **View Progress Analysis** to navigate to your Improvement Analysis page.
4. Click **Start AI Evaluation** to begin the AI Validator Interview and get a job-readiness score.

7.10 AI Evaluation (Validator Interview)

The AI Evaluation module, accessible via **Real-World Validator** in the left sidebar, conducts a structured AI interview to assess your reasoning, problem-solving ability, and job readiness. See Figure 4.7.

7.10.1 Starting a New Session

1. On the Validator page, locate the **Start New Session** panel on the right sidebar.
2. Enter your **Path ID** (numeric; you can find this in your learning path URL or be directed here automatically from the learning path completion banner).
3. Click **Create Session**. The AI will prepare an introductory question based on your full track and learning context.
4. The session status badge will update once the session is ready.

Note: If you navigated here by clicking **Start AI Evaluation** from the Learning Path page, the session is created automatically using your current path ID.

7.10.2 Conducting the AI Interview

1. when you open the AI Mentor it will show you a first question in the chat window that is on the left side.

2. read that question properly and then write your answer in the **Your answer** text area below.
3. after writing press **Enter** or click the **Send** button to send your answer. if you want to go new line without sending then use Shift+Enter for that.
4. the AI will then look at your answer and ask more questions to check that how much you really understand the topic.
5. you have to keep answering. but remember you must do atleast **3 dialogue turns** before you allowed to end the evaluation.
6. there is also a small hint showing at bottom of the chat that remind you about this minimum requirement.

7.10.3 Completing the Evaluation

1. after you finish atleast 3 dialogue turns, the **Complete Evaluation** button that is in right sidebar under **Evaluation Result** section will now become clickable.
2. click **Complete Evaluation** and then the AI will look at your whole conversation and make a scored report for you.
3. the evaluation result panel will then showing you these things:
   - **Reasoning score**: this show how logically you was solving the problems.
   - **Problem-solving score**: it show the quality of your solutions that you gave.
   - **Job readiness level**: this is like overall rating of how much you are ready for job.
   - **AI feedback**: a written feedback about how you performed in the interview.
4. click **View Full Report** if you want to open the full Evaluation Report page.
5. click **View Progress Analysis** to go to the Improvement Analysis page.

7.11 Evaluation Report

The Evaluation Report page (`/evaluation/{id}`) is where you can see all the complete results of your evaluation session.

1. this page will open automatically when evaluation is done, or you can also open it by clicking **View Full Report** from the Validator page.
2. the report will showing you these informations:

- the session and path id, and also the date when you did the evaluation.
- the scores for **Reasoning**, **Problem Solving**, and **Readiness** metrics.
- **AI Feedback**: a detailed written feedback about your overall performance.
- **Full Conversation**: you can see the complete conversation that you had with the AI during interview.

3. click **Back to Validator** if you want to go back to the Validator page.
4. click **View Progress Analysis** to see your full improvement journey that you made.

7.12 Improvement Analysis

The Improvement Analysis page (`/improvement/{pathId}`) is where you can see a full comparison of your skills from before and after the whole learning journey.

1. you can access this page from **Progress Analysis** on the Dashboard, or from the Completion Banner on Learning Path, or also from the Validator page.
2. when this page load it will show your full progress data with many things like:
   - a **AI evaluation summary** banner at the top of the page.
   - a **metrics grid** that showing the important performance numbers.
   - some **Charts** that show before and after comparison graphs.
   - a **story timeline** that show your whole learning journey.
   - **Before** and **After** skill columns with scores and levels, and you can also expand the initial assessment answers by using the **Show/Hide** toggle.
   - **Improvement percentage** is shown in the middle column where it is applicable.
   - a **Detailed Progress Analysis** written report that is generated by AI.
   - **Full Interview Transcript**: you can click the **Show** button to see the complete evaluation conversation.
3. at the bottom of the page there is two buttons:
   - click **Back to Learning Path** to go back to `/course` page.
   - click **Start New Evaluation** to go to `/validator` and do another interview.

As we can see in Figures 4.8 and 4.9, the insights dashboard is giving a good visual view of your learning

progress.

7.13 Account and Security Settings

The Account & Security page (`/account`) is accessible from the left sidebar under **Account & Security**.

7.13.1 Updating Your Profile

1. The page loads your current **Full Name** and **Email**.
2. Edit either field as needed.
3. Click **Save profile**.
4. A success message *"Profile updated successfully."* appears below the button on success.

7.13.2 Changing Your Password

1. In the **Change Password** section, enter your **Current password**.
2. Enter a **New password** (minimum 8 characters).
3. Re-enter the same new password in **Confirm new password**.
4. Click **Update password**.
5. On success, the fields are cleared and *"Password changed successfully."* is shown.

7.13.3 Managing Active Sessions

The **Active Sessions** section lists all devices currently logged into your account.

- Each session shows: session ID (truncated), device/browser info, IP address, and last activity.
- Your current session is marked with a **Current** badge.
- Click **Revoke** on any session to log it out remotely.
- Click **Revoke Others** to log out all other devices (a confirmation dialog will appear).
- Click **Revoke All** to log out every session, including the current one. You will be redirected to the Home page.
- Click **Refresh** to reload the session list.

7.13.4 Deleting Your Account

Warning: This action is permanent and cannot be undone.

1. Scroll to the **Delete Account** section.
2. Type the word **DELETE** (case-insensitive) in the confirmation field.
3. Click **Delete my account**.
4. A second browser confirmation dialog will appear. Click **OK** to confirm.
5. Your account and all data are permanently deleted. You will be redirected to the Home page.

7.14 Logging Out

To log out of GrowWise:

1. Open the **Account & Security** page from the left sidebar.
2. In the **Active Sessions** section, locate the session marked **Current**.
3. Click **Log Out This Device**.
4. You will be redirected to the Home page and the session will be terminated.

Alternatively, you can log out all devices at once by clicking **Revoke All** in the Active Sessions section.

7.15 Conclusion

This chapter provided a complete walkthrough of the GrowWise platform. A new user can follow these steps in sequence: Sign Up, Log In, Select a Track, Complete the Assessment, follow the personalised Learning Path, use the AI Mentor Chat, complete the AI Evaluation, and review their Improvement Analysis. This covers the full learning cycle from start to finish. The platform is designed to be self-explanatory, with clear guidance at every step to ensure learners can progress without confusion.

Chapter 8 Conclusions

8.1 Project Summary and Achievements

so basically in this final project we builded GrowWise which is a AI based adaptive learning platform and we think it is solving many problems that developers are facing when they try to learn new things. the main goal of this project was to make a personalized system that can give proper learning to each person individually instead of just giving same old fixed courses to everyone. when we was looking at around fifteen other platforms we found many problems in them like they dont have proper real time checking, they also dont focus much on problem solving, and there is no proper AI mentoring at all. so to fix all these things we used Next.js, FastAPI, PostgreSQL, and AI API together to build a platform that is scalable and also easy to maintain and it help people to improve their skills and thinking.

8.2 Key Findings and Results

The review demonstrated that others excel in areas such as projects (freeCodeCamp, Udacity), adaptive tests (Pluralsight, DataCamp), and AI (CYPHER Learning), but none of them combine it in the way GrowWise does. The arrangement closes the gaps with on-the-fly routes, real-time assistance with AI through RAG pulls, and complete abilities and logic examination. The modular construction allows parting work and tests, and the additions and amendments are not difficult. The complete documentation containing diagrams, interactions and plans provide proper building and rollout.

8.3 Project Scope Coverage and Objectives Achievement

We went through it all and achieved all objectives. The design manages users, path making, AI tests and tracking where necessary. It accompanies tracks, provides personal experiences and has user and admin views. All the needs concerning functionalities, performance, security, scale, and convenience are addressed. It is a combination of a method-based approach to learning and AI assistance, forming a new platform between school and work. Our fix was to the extent of what is allowed to ensure that developer learning issues were full.

8.4 Challenges Faced and Limitations

We hit some hurdles. Combining AI services and maintaining speed required special consideration and adjustments. Responsive design and backups were required to make it work well on devices and networks. The dynamic nature of AI content required powerful error corrections. Constant adjustments

were balancing between deep checks and speed. There are limitations such as the use of external AI, which might compromise the uptime and speed. It has gone web based and thus not much on mobile enthusiasts. Besides, its effectiveness is determined by the quality of materials and knowledge base.

8.5 Long-term Recommendations

there is also some ideas that we think can be done in future to make this platform even more better:

- Mobile Application Development: it will be very good if we can add iOS and Android app so that peoples can use it on their phone easily anytime.
- Advanced AI Capabilities: the AI can be make more smarter so it can do code checking, structure review, and also give custom tips based on what language the user is learning.
- Community Features: we can also add things like peer review, forums, and group projects so that users can learn together with each other.
- Industry Integration: it would be nice if we can do collaboration with real companies so users can get actual project experience and mentoring from industry peoples.
- Advanced Analytics: we can improve the analytics more by adding prediction features, gap forecasting, and other useful measures to help users keep improving.
- Multi-language Support: if we add support for more programming languages and frameworks then more developers from different background can also use this platform.
- Enterprise Features: for companies and schools we can make custom learning paths and also add team tools and tracking features for them.

8.6 Final Remarks

we believe that GrowWise will be very helpful for developers who wants to learn and improve their self. it is fixing many gaps that was existing before by using AI in a smart way and focusing on proper learning methods. the way we designed and builded this platform, we think it can handle the changes that keep happening in technology world. it also show that how AI can really change the way people learn technical things, and it can be useful for students, institutions, and also for companies. the FYP-2 plan will continue from here and we will work to make the platform more complete so it can help more developers to improve their skills in todays world.

References

[1] Crio.Do. (2025) Crio.do — project-based learning platform for developers. [Online]. Accessed: Oct. 14, 2025. [Online]. Available: https://www.crio.do/

[2] The Odin Project. (2025) The odin project — free, open-source curriculum for web development. [Online]. Accessed: Oct. 14, 2025. [Online]. Available: https://www.theodinproject.com/

[3] ProjectLearn.io. (2025) Projectlearn.io. [Online]. Accessed: Oct. 14, 2025. [Online]. Available: https://projectlearn.io/

[4] Educative. (2025) Educative — interactive courses for software developers. [Online]. Accessed: Oct. 14, 2025. [Online]. Available: https://www.educative.io/

[5] Codio. (2025) Codio — online programming platform. [Online]. Accessed: Oct. 14, 2025. [Online]. Available: https://www.codio.com/

[6] Practical Tutorials. (2025) Project-based learning repository. [Online]. Accessed: Oct. 14, 2025. [Online]. Available: https://github.com/practical-tutorials/project-based-learning

[7] Udemy. (2025) Udemy — online learning and teaching marketplace. [Online]. Accessed: Oct. 14, 2025. [Online]. Available: https://www.udemy.com/

[8] LinkedIn Learning. (2025) Linkedin learning — online courses for technology, business, and creative skills. [Online]. Accessed: Oct. 14, 2025. [Online]. Available: https://www.linkedin.com/learning/

[9] Microsoft Learn. (2025) Microsoft learn — training and documentation. [Online]. Accessed: Oct. 14, 2025. [Online]. Available: https://learn.microsoft.com/

[10] Pluralsight. (2023) Pluralsight — technology skills platform. [Online]. Accessed: Oct. 14, 2025. [Online]. Available: https://www.pluralsight.com/

[11] CYPHER Learning. (2023) Cypher learning — ai-powered learning management system. [Online]. Accessed: Oct. 14, 2025. [Online]. Available: https://www.cypherlearning.com/

[12] DataCamp. (2023) Datacamp — learn data science online. [Online]. Accessed: Oct. 14, 2025. [Online]. Available: https://www.datacamp.com/

[13] freeCodeCamp. (2023) freecodecamp — learn to code for free. [Online]. Accessed: Oct. 14, 2025. [Online]. Available: https://www.freecodecamp.org/

[14] Udacity. (2023) Udacity — online learning platform for technology. [Online]. Accessed: Oct. 14, 2025. [Online]. Available: https://www.udacity.com/

[15] CodeCombat. (2023) Codecombat — learn to code through gameplay. [Online]. Accessed: Oct. 14, 2025. [Online]. Available: https://codecombat.com/